

# printquote

Presupuesta una impresión 3D en segundos: arrastra un STL, míralo en 3D y obtén peso, tiempo estimado y precio. Tu archivo no sale de tu navegador.

|               |                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------|
| Repositorio   | <a href="https://github.com/BertMarti/printquote">https://github.com/BertMarti/printquote</a> |
| Web publicada | <a href="https://bertmarti.github.io/printquote/">https://bertmarti.github.io/printquote/</a> |
| Stack         | TypeScript estricto · Vite · three.js · Vitest · ESLint · GitHub Actions · GitHub Pages       |
| Fecha         | 1 de octubre de 2026                                                                          |
| Autoría       | Alberto Martínez, con un equipo de agentes de IA (Claude Code + OpenCode)                     |

## Contenido

1. Qué es y para quién
  2. Funcionamiento a nivel de usuario
  3. Funcionamiento a nivel técnico
  4. Flujo de un presupuesto
  5. Funcionalidades
  6. Calidad, CI/CD y equipo
  7. Novedades de la noche del 30/09 al 01/10
  8. Cómo explotarlo económicamente
  9. Siguiendo pasos y acciones necesarias por ti, Alberto
- Anexo: código fuente

## Qué es y para quién

**printquote** es una web que calcula el presupuesto de una pieza de impresión 3D. Arrastras un archivo STL, lo ves en 3D sobre la cama de la impresora y obtienes al instante su volumen, peso, tiempo estimado y precio final con el desglose de material, energía y margen.

Todo ocurre en el navegador: no hay servidor, ni cuentas, ni analítica. El archivo nunca se sube a ningún sitio.

| Perfil                       | Para qué lo usa                                                               |
|------------------------------|-------------------------------------------------------------------------------|
| Maker con impresora          | Saber cuánto le cuesta de verdad una pieza antes de imprimirla o de venderla. |
| Pequeño negocio de impresión | Dar presupuestos rápidos y coherentes a sus clientes, con margen incluido.    |
| Cliente final                | Hacerse una idea del coste de un modelo descargado antes de encargarlo.       |
| Docente o estudiante         | Entender la relación entre geometría, relleno, material y coste.              |

## Funcionamiento a nivel de usuario

The screenshot shows the printquote web interface. On the left, a 3D model of a piece is displayed on a grid. The grid has axes labeled X, Y, and Z. The piece is a simple, angular shape. On the right, there is a sidebar with technical specifications and pricing. The sidebar is divided into sections: 01 PIEZA, 02 MATERIAL, and 03 IMPRESIÓN. The 01 PIEZA section shows the volume (123,83 cm³), surface (320,78 cm²), dimensions (70,0 x 90,0 x 88,0 mm), and number of triangles (56). The 02 MATERIAL section shows a table with columns for material (PLA, PETG, ABS, TPU) and price (1,24, 1,27, 1,04, 1,21). The 03 IMPRESIÓN section shows the fill percentage (20%), perimeter (2 uds.), line width (0,45 mm), and volumetric flow (8 mm³/s). At the bottom, the total price is displayed as 1,58 €.

| 01 ■ PIEZA            |                       |
|-----------------------|-----------------------|
| Volumen               | 123,83 cm³            |
| Superficie            | 320,78 cm²            |
| Dimensiones X x Y x Z | 70,0 x 90,0 x 88,0 mm |
| Triángulos            | 56                    |

| 02 ■ MATERIAL |      |
|---------------|------|
| PLA           | 1,24 |
| PETG          | 1,27 |
| ABS           | 1,04 |
| TPU           | 1,21 |

DENSIDAD EN G/CM³

Precio del material 20 €/kg

| 03 ■ IMPRESIÓN     |         |
|--------------------|---------|
| Relleno            | 20 %    |
| Perímetros         | 2 uds.  |
| Ancho de línea     | 0,45 mm |
| Caudal volumétrico | 8 mm³/s |

Cama X 220 Y 220 Z 250 mm

TOTAL 1,58 €

1 copia - PLA 20 % - 1 h 45 min (estimación)

Copiar presupuesto Imprimir

printquote con la pieza de ejemplo: visor 3D y ficha técnica con el presupuesto en vivo.

1. Abre <https://bertmarti.github.io/printquote/>. Para verlo con una pieza ya cargada: [.../printquote/#ejemplo](https://bertmarti.github.io/printquote/#ejemplo).
2. Arrastra un STL a cualquier parte de la ventana o pulsa **Abrir STL**. Si no tienes ninguno, pulsa **Probar con pieza de ejemplo**.
3. Gira la pieza arrastrando, acércala con la rueda y desplázala con el botón derecho. **Restablecer vista** la vuelve a encuadrar.
4. En la ficha de la derecha revisa la sección **01 Pieza**: volumen, superficie, dimensiones y triángulos. Si la malla está abierta o no cabe en la cama, aparece un aviso.

5. Elige el **material** (PLA, PETG, ABS o TPU) y ajusta su precio por kilo.
6. Ajusta la **impresión**: relleno, perímetros, ancho de línea, caudal y tamaño de la cama.
7. Ajusta **energía y negocio**: potencia media, precio del kWh, margen y número de copias.
8. El **total** se actualiza en vivo en el bloque inferior, con el tiempo rotulado como estimación.
9. Pulsa **Copiar presupuesto** para pegarlo en un mensaje o **Imprimir** para obtener una hoja limpia (o un PDF) con la vista 3D y el desglose.

Los ajustes se recuerdan en tu navegador para la próxima vez. La web cambia sola a modo oscuro si tu sistema lo usa, y se puede manejar entera con el teclado (↑/↓ ajustan los números; con Mayús, de 10 en 10).

# Funcionamiento a nivel técnico

## Arquitectura

| Módulo                                           | Responsabilidad                                                                                                                                                                                |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/stl/parse.ts                                 | Parser STL propio. Distingue binario y ASCII por tamaño (84 + 50·n bytes) y contenido, no solo por la cabecera «solid». Errores claros en español.                                             |
| src/stl/analyze.ts · analyzer.ts · stl.worker.ts | Lectura y análisis en un Web Worker para no bloquear la interfaz con archivos grandes; si el worker falla, se hace en el hilo principal. Descarta resultados viejos si se suelta otro archivo. |
| src/stl/geometry.ts                              | Volumen (suma de tetraedros con signo), área, caja envolvente, triángulos, aristas abiertas y normales invertidas.                                                                             |
| src/quote/                                       | Modelo de coste como funciones puras: materiales, cáscara, relleno, peso, tiempo, energía, margen y copias; formato es-ES; texto del presupuesto.                                              |
| src/viewer/viewer.ts                             | Escena three.js: cama con rejilla de 10 mm, cámara con OrbitControls, luces sobrias, material mate y aristas.                                                                                  |
| src/ui/                                          | Arranque de la app, campos numéricos accesibles, almacenamiento en localStorage y hoja de impresión.                                                                                           |
| scripts/generate-sample.mjs                      | Genera la pieza de ejemplo original (soporte de móvil low-poly) como STL binario.                                                                                                              |
| src/viewer/webgl.ts                              | Carga del visor three.js bajo demanda (import dinámico): el JavaScript inicial pasa de 580 kB a 26 kB.                                                                                         |

## Cómo se calcula

grosor de pared = perímetros × ancho de línea  
cáscara = mín(área × grosor de pared, volumen)  
relleno = relleno% × (volumen - cáscara)  
volumen impreso = cáscara + relleno (por copia)

peso (g) = volumen impreso (cm³) × densidad (g/cm³)  
coste material = peso (kg) × precio (€/kg)  
tiempo (h) = volumen impreso / caudal (mm³/s) + 5 min ← estimación  
energía (kWh) = potencia media (W) × tiempo (h) / 1000  
coste energía = energía × precio (€/kWh)

subtotal = (coste material + coste energía) × copias  
TOTAL = subtotal × (1 + margen%)

**Ejemplo comprobado en los tests:** un cubo de 20 mm tiene 8.000 mm³ de volumen y 2.400 mm² de superficie. En PLA con los valores por defecto sale una cáscara de 2.160 mm³ más 1.168 mm³ de relleno: 4,13 g, 12 minutos estimados y 0,10 € en total (cada importe se redondea a céntimos y el total es la suma de las líneas, como en una factura).

**Límites honestos:** no se tienen en cuenta soportes, balsa, purga ni altura de capa, y la cáscara se aproxima como área × grosor. El tiempo es siempre una estimación: la cifra real la da el laminador.

## Decisiones de diseño

- **Sin servidor:** coste cero de alojamiento (GitHub Pages) y privacidad total.
- **Parser propio** en lugar de un cargador genérico: permite errores claros, detección robusta del formato y tests con STL contruidos en memoria.
- **Campos de texto con inputmode decimal** en lugar de type=number, para aceptar coma o punto decimal igual en todos los navegadores.

- **El naranja nunca como texto sobre fondo claro** (contraste 2,85:1): se usa en rellenos y en la cifra del total sobre fondo tinta, para cumplir AA.
- **base: /printquote/** también en desarrollo, para que el servidor local y el de vista previa se comporten igual que Pages.

## Flujo de un presupuesto



## Funcionalidades

| Funcionalidad           | Detalle                                                                                        |
|-------------------------|------------------------------------------------------------------------------------------------|
| Carga de STL            | Binario y ASCII; arrastrar y soltar o selector de archivos.                                    |
| Pieza de ejemplo        | Soporte de móvil low-poly original; enlace directo con #ejemplo.                               |
| Mediciones              | Volumen, superficie, dimensiones $X \times Y \times Z$ y número de triángulos.                 |
| Avisos                  | Malla abierta, normales invertidas y pieza que no cabe en la cama (permite girar 90° sobre Z). |
| Visor 3D                | three.js con rejilla de cama, órbita, zoom, desplazamiento y restablecer vista; modo oscuro.   |
| Materiales              | PLA, PETG, ABS y TPU con densidad y precio por kilo editable.                                  |
| Parámetros de impresión | Relleno, perímetros, ancho de línea, caudal volumétrico y tamaño de cama.                      |
| Energía y negocio       | Potencia media, precio del kWh, margen y copias.                                               |
| Presupuesto en vivo     | Desglose y total recalculados al instante; tiempo rotulado como estimación.                    |
| Copiar e imprimir       | Presupuesto en texto plano y hoja de impresión limpia con la vista 3D.                         |
| Memoria de ajustes      | localStorage del navegador, con manejo de errores.                                             |
| Accesibilidad           | Etiquetas, foco visible, teclado completo, contraste AA, diseño adaptable a móvil.             |

## Calidad, CI/CD y equipo

- **Tests (Vitest):** parser binario y ASCII con buffers en memoria, geometría (cubo de 20 mm: 8.000 mm<sup>3</sup> y 2.400 mm<sup>2</sup>), archivos corruptos, pieza de ejemplo cerrada y modelo de coste (relleno 0 % y 100 %, copias y margen).
- **CI** (ci.yml): en cada PR y en main, en Ubuntu y Windows: npm ci, lint, tests y build.
- **CD** (deploy.yml): al actualizar main, construye y publica en GitHub Pages.
- **Git:** main protegida (solo por PR), commits convencionales y una rama por agente: agent/builder, agent/qa y agent/opencode-docs.
- **Contexto para IA:** AGENTS.md, CLAUDE.md, opencode.json y MEMORY.md, que cada agente actualiza al terminar.
- **Revisión QA** (Claude Opus, 132 tests): STL con BOM, nombres de sólido con palabras clave, números fuera de float32, binarios con recuento 0, ASCII grandes (de ~800 MB de memoria a un tokenizador byte a byte), soldado de vértices con tolerancia, Web Worker, visor bajo demanda, desglose que no sumaba el total, avisos accesibles con regiones vivas, Open Graph y aviso de pieza en metros o pulgadas.
- **Documentación** (Claude Sonnet): docs/USO.md con 14 secciones, ejemplos de cálculo comprobados con la pieza de ejemplo y guía de calibración con el laminador, y CONTRIBUTING.md.
- **Pull requests:** #1 builder, #2 qa (sobre builder) y #3 docs (sobre qa), todos con el CI en verde.

## Novedades de la noche del 30/09 al 01/10

- **v0.3.0 (publicada):** PDF con fuentes incrustadas (Noto Sans y JetBrains Mono, OFL, recortadas a 224 kB) que admite latín extendido, griego y cirílico, cargadas solo al descargar el PDF; metadatos para redes en inglés; aviso en OBJ con polígonos de más de 200 vértices; tope de 6 millones de triángulos en 3MF; zoom con + y -; capturas nuevas; licencias OFL servidas en la web.
- **v0.4.0 «Pro» (publicada):** app instalable que funciona sin conexión (PWA con caché versionada); historial de presupuestos en el navegador con exportación CSV protegida contra inyección de fórmulas; botón «Copiar enlace» que comparte los parámetros (nunca el archivo ni el cliente).
- **Calidad:** más de 300 tests, CI en Ubuntu y Windows, revisión de cada PR por un agente code-reviewer y comprobación en la web publicada (total 1,59 € con la pieza de ejemplo).

## Cómo explotarlo económicamente

El nicho de la impresión 3D tiene mucha gente que vende piezas y poca herramienta sencilla y bonita para presupuestar. Ideas ordenadas de menor a mayor esfuerzo:

| Idea                        | Cómo                                                                                                                      | Esfuerzo |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------|----------|
| Afiliación de filamento     | Enlaces de afiliado a cada material (Amazon, tiendas de filamento) en el selector de material.                            | Bajo     |
| Perfiles de impresora       | Perfiles precargados (Bambu, Prusa, Creality...) con caudal y potencia reales. Gratis los básicos, pack premium.          | Bajo     |
| Presupuesto con tu marca    | Versión Pro para negocios: logotipo, datos fiscales, IVA, numeración y PDF de presupuesto. Pago único o suscripción baja. | Medio    |
| Widget para tiendas         | Un fragmento que las tiendas de impresión 3D pegan en su web para que sus clientes suban un STL y vean el precio.         | Medio    |
| Marketplace de encargos     | Conectar a quien tiene un STL con makers cercanos que lo imprimen, cobrando comisión.                                     | Alto     |
| Integración con laminadores | Leer G-code o 3MF para dar el tiempo y el filamento reales en lugar de estimaciones.                                      | Medio    |

El primer paso barato: publicar la herramienta en comunidades de impresión 3D (Reddit r/3Dprinting, foros en español, grupos de Telegram), medir visitas y preguntar qué echan en falta.

## Siguientes pasos y acciones necesarias por ti, Alberto

### Acciones necesarias por ti

1. **Fusionar los PR en orden** (builder → qa → docs). El sistema de permisos no me deja fusionar en main. Al fusionar, se publica la web automáticamente.
2. Comprobar la web publicada: <https://bertmarti.github.io/printquote/#ejemplo>
3. Probar con STL reales tuyos y comparar el tiempo y el peso con lo que dice tu laminador para ajustar los valores por defecto (caudal y sobrecarga).

### Mejoras técnicas propuestas

- Soporte de 3MF y OBJ además de STL.
- Estimación de soportes según voladizos y orientación automática óptima.
- Varios objetos a la vez y presupuesto por lote.

- Traducción al inglés para ampliar audiencia.

## Anexo: código fuente

Archivos principales tal y como están en main al generar este documento. Las líneas muy largas se parten y continúan en la línea siguiente.

### package.json · 39 líneas

```
{
  "name": "printquote",
  "private": true,
  "version": "0.4.0",
  "description": "Presupuesta una impresión 3D en el navegador a partir de un STL.",
  "type": "module",
  "license": "MIT",
  "engines": {
    "node": ">=22.12"
  },
  "scripts": {
    "dev": "vite",
    "build": "npm run typecheck && vite build",
    "preview": "vite preview",
    "lint": "eslint .",
    "test": "vitest run",
    "test:watch": "vitest",
    "sample": "node scripts/generate-sample.mjs",
    "typecheck": "tsc --noEmit && tsc --noEmit -p tests"
  },
  "dependencies": {
    "@pdf-lib/fontkit": "^1.1.1",
    "pdf-lib": "^1.17.1",
    "three": "^0.186.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/node": "^22.20.4",
    "@types/three": "^0.186.0",
    "eslint": "^10.11.0",
    "globals": "^17.12.0",
    "happy-dom": "^20.14.5",
    "typescript": "^6.0.3",
    "typescript-eslint": "^8.71.0",
    "vite": "^8.3.1",
    "vitest": "^5.0.2"
  }
}
```

### vite.config.ts · 21 líneas

```
import { defineConfig } from 'vitest/config';
import { pwa } from './scripts/pwa-plugin';

// La web se publica en GitHub Pages bajo /printquote/. Se usa la misma base en
// desarrollo, build y preview para que las rutas se comporten igual en todas partes.
export default defineConfig({
  base: '/printquote/',
  plugins: [pwa()],
  build: {
    target: 'es2022',
    // Dos chunks pesados y esperados, ambos con import(): el visor (three.js, ~560 kB) tras pintar la
    // interfaz, y el PDF (pdf-lib + fontkit, ~1,1 MB) solo al pulsar «Descargar PDF».
    chunkSizeWarningLimit: 1200,
  },
  test: {
    include: ['tests/**/*.test.ts'],
    setupFiles: ['tests/setup.ts'],
    environment: 'node',
  },
});
```

### src/main.ts · 7 líneas

```
import './styles.css';
import { registerServiceWorker } from './pwa';
import { startApp } from './ui/app';

startApp();
registerServiceWorker();
```

### src/stl/types.ts · 40 líneas

```
/** Formato de origen del archivo: STL binario o ASCII, OBJ o 3MF. */
export type MeshFormat = 'binary' | 'ascii' | 'obj' | '3mf';

/**
 * Malla triangular sin indexar tal como viene en el archivo.
 * `positions` guarda 9 números por triángulo (x, y, z de sus tres vértices), en mm.
 */
export interface Mesh {
  readonly positions: Float32Array;
  readonly triangleCount: number;
  readonly format: MeshFormat;
  /** Solo OBJ: polígonos de más de 200 vértices, triangulados en abanico aunque fueran cóncavos. */
  readonly largePolygons?: number;
}

export interface Vec3 {
  readonly x: number;
  readonly y: number;
  readonly z: number;
}

export interface BoundingBox {
  readonly min: Vec3;
  readonly max: Vec3;
  readonly size: Vec3;
}

export interface MeshStats {
  readonly triangleCount: number;
  /** Volumen en mm³ (valor absoluto). */
  readonly volume: number;
  /** Volumen con signo antes del valor absoluto. Negativo o = 0 indica malla abierta o invertida. */
  readonly signedVolume: number;
  /** Área de superficie en mm². */
  readonly surfaceArea: number;
  readonly bounds: BoundingBox;
  /** Aristas que no comparten exactamente dos triángulos (bordes abiertos o no variedad). */
  readonly openEdges: number;
}
```

### src/stl/parse.ts · 246 líneas

```
import type { ErrorKey } from '../i18n/es';
import type { Params } from '../i18n/interpolate';
import { ModelParseError } from './errors';
import type { Mesh } from './types';

/** Error de lectura de un STL (mensajes en `i18n/es.ts`, claves `err.stl.*`). */
export class StlParseError extends ModelParseError {
  constructor(code: ErrorKey, params?: Params) {
    super(code, params);
    this.name = 'StlParseError';
  }
}

const HEADER_BYTES = 80;
const COUNT_BYTES = 4;
const BINARY_PREAMBLE = HEADER_BYTES + COUNT_BYTES;
const BYTES_PER_TRIANGLE = 50;
/** Tamaño de la muestra que se inspecciona para decidir si el archivo es texto. */
const TEXT_SNIFF_BYTES = 1024;
/** Marca de orden de bytes de UTF-8 que algunos editores de Windows añaden al principio. */
const UTF8_BOM = [0xef, 0xbb, 0xbf] as const;
/** Una coordenada ASCII más larga que esto no es un número razonable. */
const MAX_NUMBER_TOKEN = 64;

/**
 * Lee un STL binario o ASCII.
 *
 * La detección no se fía solo de la cabecera «solid»: muchos exportadores escriben
 * «solid ...» en la cabecera de archivos binarios. Se decide así:
 * 1. Si el tamaño coincide exactamente con 84 + 50 × nº de triángulos, es binario,
 *    salvo que empiece por «solid», sea texto y contenga «facet» (ASCII casual).
 * 2. Si parece texto y empieza por «solid», es ASCII.
 * 3. Si el tamaño es mayor de lo esperado (relleno al final), se acepta como binario.
 * 4. Si el recuento es 0 pero el tamaño cuadra con 84 + 50 × n, se leen esos n
 *    triángulos (hay exportadores que no rellenan el recuento).
 */
export function parseStl(data: ArrayBuffer | Uint8Array): Mesh {
  const bytes = data instanceof Uint8Array ? data : new Uint8Array(data);

  if (bytes.byteLength === 0) {
    throw new StlParseError('err.empty');
  }

  const looksAscii = startsWithSolid(bytes) && isMostlyText(bytes);

  if (bytes.byteLength >= BINARY_PREAMBLE) {
    const view = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
    const count = view.getUint32(HEADER_BYTES, true);
    const expected = BINARY_PREAMBLE + count * BYTES_PER_TRIANGLE;

    if (expected === bytes.byteLength && !(looksAscii && containsFacet(bytes))) {
      return parseBinary(view, count);
    }
    if (looksAscii) {
      return parseAscii(bytes);
    }
    if (count > 0 && expected < bytes.byteLength) {
      return parseBinary(view, count);
    }
    const inferred = (bytes.byteLength - BINARY_PREAMBLE) / BYTES_PER_TRIANGLE;
    if (count === 0 && Number.isInteger(inferred)) {
      return parseBinary(view, inferred);
    }
    throw new StlParseError('err.stl.size');
  }

  if (looksAscii) {
    return parseAscii(bytes);
  }
  throw new StlParseError('err.stl.small');
}

function parseBinary(view: DataView, count: number): Mesh {
  if (count === 0) {
    throw new StlParseError('err.stl.noTriangles');
  }
}
```

```

const positions = new Float32Array(count * 9);
for (let t = 0; t < count; t++) {
  // Cada triángulo: normal (12 bytes), 3 vértices (36 bytes), atributo (2 bytes).
  const base = BINARY_PREAMBLE + t * BYTES_PER_TRIANGLE + 12;
  for (let k = 0; k < 9; k++) {
    const value = view.getFloat32(base + k * 4, true);
    if (!Number.isFinite(value)) {
      throw new StlParseError('err.stl.badTriangle', { n: t + 1 });
    }
    positions[t * 9 + k] = value;
  }
}
return { positions, triangleCount: count, format: 'binary' };
}

/**
 * Recorre el texto byte a byte, sin decodificarlo entero ni partirlo en un array de
 * palabras: con archivos de cientos de MB eso multiplicaba la memoria por diez.
 */
class Tokenizer {
  pos: number;
  start = 0;
  end = 0;

  constructor(private readonly bytes: Uint8Array, from: number) {
    this.pos = from;
  }

  /** Avanza a la siguiente palabra. Cualquier byte ≤ 0x20 (espacio, tabulador, CR, LF...) separa. */
  next(): boolean {
    const { bytes } = this;
    let i = this.pos;
    while (i < bytes.length && (bytes[i] ?? 0) <= 0x20) i++;
    if (i >= bytes.length) {
      this.pos = i;
      return false;
    }
    this.start = i;
    while (i < bytes.length && (bytes[i] ?? 0) > 0x20) i++;
    this.end = this.pos = i;
    return true;
  }

  /** ¿La palabra actual es `word` (en minúsculas), sin distinguir mayúsculas? */
  is(word: string): boolean {
    if (this.end - this.start !== word.length) return false;
    for (let k = 0; k < word.length; k++) {
      // `| 0x20` pasa A-Z a minúsculas; `word` solo contiene letras minúsculas.
      if (((this.bytes[this.start + k] ?? 0) | 0x20) !== word.charCodeAt(k)) return false;
    }
    return true;
  }

  /** La palabra actual como número; NaN si no tiene forma de número decimal (nada de «0x10» ni «inf»). */
  number(): number {
    if (this.end - this.start > MAX_NUMBER_TOKEN) return Number.NaN;
    let text = '';
    for (let i = this.start; i < this.end; i++) {
      const b = this.bytes[i] ?? 0;
      const isNumberChar = (b >= 0x30 && b <= 0x39) || b === 0x2e || b === 0x2b || b === 0x2d || b === 0x65 || b ===
0x45;
      if (!isNumberChar) return Number.NaN;
      text += String.fromCharCode(b);
    }
    return Number(text);
  }

  /** Salta el resto de la línea (el nombre tras «solid»/«endsolid»), salvo que no haya más saltos de línea. */
  skipLine(): void {
    const { bytes } = this;
    for (let i = this.pos; i < bytes.length; i++) {
      if (bytes[i] === 0x0a || bytes[i] === 0x0d) {
        this.pos = i;
        return;
      }
    }
  }
}
// Archivo en una sola línea: no se puede saber dónde acaba el nombre; se sigue leyendo.

```

```

    }
}

function parseAscii(bytes: Uint8Array): Mesh {
    const tokens = new Tokenizer(bytes, hasBom(bytes) ? UTF8_BOM.length : 0);
    let values = new Float32Array(9 * 1024);
    let length = 0;
    let facet = 0;
    let verticesInFacet = 0;
    let insideFacet = false;

    while (tokens.next()) {
        if (tokens.is('vertex')) {
            if (!insideFacet) {
                throw new StlParseError('err.stl.vertexOutside');
            }
            if (verticesInFacet === 3) {
                throw new StlParseError('err.stl.facetVertices', { n: facet });
            }
            if (length + 3 > values.length) {
                const grown = new Float32Array(values.length * 2);
                grown.set(values);
                values = grown;
            }
            for (let k = 0; k < 3; k++) {
                const value = tokens.next() ? tokens.number() : Number.NaN;
                // Math.fround: 1e39 es finito en JS, pero no cabe en el float de 32 bits en que se guarda.
                if (!Number.isFinite(Math.fround(value))) {
                    throw new StlParseError('err.stl.badVertex', { n: facet });
                }
                values[length++] = value;
            }
            verticesInFacet++;
        } else if (tokens.is('facet')) {
            if (insideFacet) {
                throw new StlParseError('err.stl.facetOpen', { n: facet });
            }
            insideFacet = true;
            facet++;
            verticesInFacet = 0;
        } else if (tokens.is('endfacet')) {
            if (!insideFacet || verticesInFacet !== 3) {
                throw new StlParseError('err.stl.facetVertices', { n: facet });
            }
            insideFacet = false;
        } else if (tokens.is('solid') || tokens.is('endsolid')) {
            // El nombre del sólido puede contener cualquier cosa, incluso «facet» o «vertex».
            tokens.skipline();
        }
        // El resto («normal», «outer», «loop», «endloop» y sus números) no hace falta:
        // las normales se recalculan a partir de los vértices.
    }

    if (insideFacet) {
        throw new StlParseError('err.stl.truncated');
    }
    if (facet === 0) {
        throw new StlParseError('err.stl.noTriangles');
    }

    return { positions: values.slice(0, length), triangleCount: facet, format: 'ascii' };
}

function hasBom(bytes: Uint8Array): boolean {
    return bytes[0] === UTF8_BOM[0] && bytes[1] === UTF8_BOM[1] && bytes[2] === UTF8_BOM[2];
}

function startsWithSolid(bytes: Uint8Array): boolean {
    let i = hasBom(bytes) ? UTF8_BOM.length : 0;
    while (i < bytes.length && isWhitespace(bytes[i] ?? 0)) i++;
    const word = String.fromCharCode(...bytes.subarray(i, i + 5)).toLowerCase();
    return word === 'solid';
}

/** Espacio, tabulador, LF, tabulador vertical, salto de página o CR. */
function isWhitespace(byte: number): boolean {
    return byte === 0x20 || (byte >= 0x09 && byte <= 0x0d);
}

```

```

}

/** Texto = sin bytes nulos ni de control (salvo espacios en blanco) en la muestra inicial. */
function isMostlyText(bytes: Uint8Array): boolean {
  const end = Math.min(bytes.length, TEXT_SNIFF_BYTES);
  for (let i = 0; i < end; i++) {
    const b = bytes[i] ?? 0;
    if (b < 0x20 && !isWhitespace(b)) return false;
  }
  return true;
}

function containsFacet(bytes: Uint8Array): boolean {
  const sample = String.fromCharCode(...bytes.subarray(0, TEXT_SNIFF_BYTES)).toLowerCase();
  return sample.includes('facet');
}

```

## src/stl/obj.ts · 216 líneas

```
import type { ErrorKey } from '../i18n/es';
import type { Params } from '../i18n/interpolate';
import { ModelParseError } from './errors';
import type { Mesh } from './types';

/** Un OBJ con un byte nulo en el arranque es un binario cualquiera, no texto. */
const SNIFF_BYTES = 1024;

const NUMBER = /^[+]?(\d+\.\d*|\.\d+)([eE][+]?[d+])?$/;
const INTEGER = /^[+]?[d+)$/;

/** Vector de números que crece por duplicación (evita `push` sobre millones de valores). */
class Growable<T extends Float32Array | Int32Array> {
  data: T;
  length = 0;

  constructor(private readonly make: (size: number) => T, initial = 3 * 1024) {
    this.data = make(initial);
  }

  push(value: number): void {
    if (this.length === this.data.length) {
      const grown = this.make(this.data.length * 2);
      grown.set(this.data);
      this.data = grown;
    }
    this.data[this.length++] = value;
  }
}

/** Polígonos de más vértices que esto se triangulan en abanico aunque sean cóncavos (el recorte de orejas es  $O(n^3)$ ).
 */
export const MAX_EAR_CLIPPING = 200;

/**
 * Triangula un polígono (índices absolutos en `vertices`). Convexo: abanico desde el primer
 * vértice. Cóncavo: recorte de orejas sobre el plano de mayor área proyectada (normal de Newell).
 * Si el polígono es degenerado, referencia vértices que aún no se han leído o es enorme, cae al abanico.
 * ponytail: los polígonos cóncavos de más de 200 vértices se siguen trianguyendo en abanico; `parseObj`
 * los cuenta en `largePolygons` y la interfaz avisa.
 */
export function triangulate(
  face: readonly number[],
  vertices: Float32Array,
  vertexCount: number,
  emit: (a: number, b: number, c: number) => void,
): void {
  const n = face.length;
  const fan = (): void => {
    for (let k = 1; k + 1 < n; k++) emit(face[0] ?? 0, face[k] ?? 0, face[k + 1] ?? 0);
  };
  if (n <= 3 || n > MAX_EAR_CLIPPING || face.some((v) => v >= vertexCount)) return fan();

  const at = (i: number, axis: number): number => vertices[(face[i] ?? 0) * 3 + axis] ?? 0;
  // Normal de Newell: robusta con polígonos cóncavos y vértices colineales.
  let nx = 0;
  let ny = 0;
  let nz = 0;
  for (let i = 0; i < n; i++) {
    const j = (i + 1) % n;
    nx += (at(i, 1) - at(j, 1)) * (at(i, 2) + at(j, 2));
    ny += (at(i, 2) - at(j, 2)) * (at(i, 0) + at(j, 0));
    nz += (at(i, 0) - at(j, 0)) * (at(i, 1) + at(j, 1));
  }
  const largest = Math.max(Math.abs(nx), Math.abs(ny), Math.abs(nz));
  if (!largest > 0) return fan();
  // Se descarta el eje de la normal mayor; (u, v) son los otros dos en orden cíclico, de modo que
  // con la normal positiva el polígono queda en sentido antihorario.
  const drop = Math.abs(nz) === largest ? 2 : Math.abs(nx) === largest ? 0 : 1;
  const sign = (drop === 0 ? nx : drop === 1 ? ny : nz) > 0 ? 1 : -1;
  const uAxis = (drop + 1) % 3;
  const vAxis = (drop + 2) % 3;
  const u = (i: number): number => at(i, uAxis);
  const v = (i: number): number => at(i, vAxis);
  /** > 0 si el giro a→b→c va hacia el lado de la normal (esquina convexa). */
  const turn = (a: number, b: number, c: number): number =>
```

```

    sign * ((u(b) - u(a)) * (v(c) - v(a)) - (v(b) - v(a)) * (u(c) - u(a)));

const ring = Array.from({ length: n }, (_, i) => i);
if (ring.every((b, i) => turn(ring[(i + n - 1) % n] ?? 0, b, ring[(i + 1) % n] ?? 0) >= 0)) return fan();

const same = (p: number, q: number): boolean => u(p) === u(q) && v(p) === v(q);
const insideTriangle = (p: number, a: number, b: number, c: number): boolean =>
    !same(p, a) && !same(p, b) && !same(p, c) && turn(a, b, p) >= 0 && turn(b, c, p) >= 0 && turn(c, a, p) >= 0;

while (ring.length > 3) {
    const m = ring.length;
    let ear = -1;
    for (let i = 0; i < m && ear < 0; i++) {
        const a = ring[(i + m - 1) % m] ?? 0;
        const b = ring[i] ?? 0;
        const c = ring[(i + 1) % m] ?? 0;
        if (turn(a, b, c) < 0) continue;
        if (ring.every((p) => p === a || p === b || p === c || !insideTriangle(p, a, b, c))) ear = i;
    }
    // Sin orejas (polígono que se cruza a sí mismo): se corta por donde sea para terminar siempre.
    if (ear < 0) ear = 0;
    emit(face[ring[(ear + m - 1) % m] ?? 0] ?? 0, face[ring[ear] ?? 0] ?? 0, face[ring[(ear + 1) % m] ?? 0] ?? 0);
    ring.splice(ear, 1);
}
emit(face[ring[0] ?? 0] ?? 0, face[ring[1] ?? 0] ?? 0, face[ring[2] ?? 0] ?? 0);
}

/**
 * Lee un OBJ de texto (Wavefront): vértices `v` y caras `f`.
 * - Caras de más de 3 vértices: en abanico si son convexas (lo habitual) y por recorte de orejas
 *   si son cóncavas (ver `triangulate`).
 * - Índices negativos: relativos al último vértice leído hasta ese punto de la línea.
 * - Se ignoran normales, texturas, grupos, materiales, líneas y puntos.
 * - Todas las piezas del archivo (`o`/`g`) se juntan en una sola malla.
 */
export function parseObj(data: ArrayBuffer | Uint8Array): Mesh {
    const bytes = data instanceof Uint8Array ? data : new Uint8Array(data);
    if (bytes.byteLength === 0) throw new ModelParseError('err.empty');
    for (let i = 0, end = Math.min(bytes.length, SNIFF_BYTES); i < end; i++) {
        if (bytes[i] === 0) throw new ModelParseError('err.obj.binary');
    }

    const text = new TextDecoder('utf-8').decode(bytes);
    const vertices = new Growable((n) => new Float32Array(n));
    const faceIndices = new Growable((n) => new Int32Array(n)); // 3 por triángulo, ya absolutos
    let vertexCount = 0;
    let triangleCount = 0;
    let largePolygons = 0;
    let lineNumber = 0;

    const fail = (code: ErrorKey, params: Params = {}): never => {
        throw new ModelParseError(code, { line: lineNumber, ...params });
    };

    let pos = 0;
    while (pos < text.length) {
        // Una línea acabada en «\» continúa en la siguiente (algunos exportadores parten las caras largas).
        let line = '';
        for (;;) {
            let nl = text.indexOf('\n', pos);
            if (nl < 0) nl = text.length;
            const part = text.slice(pos, nl);
            pos = nl + 1;
            lineNumber++;
            const trimmed = part.trimEnd();
            if (trimmed.endsWith('\\\')) {
                // Un «\» final sin nada detrás (fin del archivo) simplemente se descarta.
                line += trimmed.slice(0, -1) + ' ';
                if (pos >= text.length) break;
            } else {
                line += trimmed;
                break;
            }
        }

        const hash = line.indexOf('#');
        if (hash >= 0) line = line.slice(0, hash);
    }

```

```

line = line.trim();
if (line === '') continue;

const first = line.charCodeAt(0);
// Solo interesan `v` y `f` (seguidas de espacio): `vt`, `vn`, `vp`, `o`, `g`, `s`, `l`... se saltan.
if ((first !== 0x76 && first !== 0x66) || !/\s/.test(line.charAt(1))) continue;
const tokens = line.split(/\s+/);

if (first === 0x76) {
  // v x y z [w] (algunos exportadores añaden color: v x y z r g b)
  if (tokens.length < 4) fail('err.obj.vertexCoords');
  for (let k = 1; k <= 3; k++) {
    const token = tokens[k] ?? '';
    const value = NUMBER.test(token) ? Number(token) : Number.NaN;
    // Math.fround: 1e39 es finito en JS, pero no cabe en el float de 32 bits en que se guarda.
    if (!Number.isFinite(Math.fround(value))) fail('err.obj.badCoord', { token: token.slice(0, 20) });
    vertices.push(value);
  }
  vertexCount++;
} else {
  // f v1[/vt1[/vn1]] v2 v3 ...
  if (tokens.length < 4) fail('err.obj.faceVertices');
  const face: number[] = [];
  for (let k = 1; k < tokens.length; k++) {
    const ref = (tokens[k] ?? '').split('/')[0] ?? '';
    if (!INTEGER.test(ref)) fail('err.obj.badIndex', { token: (tokens[k] ?? '').slice(0, 20) });
    const index = Number(ref);
    if (index === 0) fail('err.obj.zeroIndex');
    // Positivo: 1 = primer vértice. Negativo: -1 = el último leído hasta ahora.
    const absolute = index > 0 ? index - 1 : vertexCount + index;
    if (absolute < 0) fail('err.obj.beforeFirst', { index });
    // Los índices se guardan en un Int32Array: uno enorme se cortarí y apuntaría a un vértice cualquiera.
    if (absolute > 0x7fffffff) fail('err.obj.outOfRange', { index, count: vertexCount });
    face.push(absolute);
  }
  if (face.length > MAX_EAR_CLIPPING) largePolygons++;
  triangulate(face, vertices.data, vertexCount, (a, b, c) => {
    faceIndices.push(a);
    faceIndices.push(b);
    faceIndices.push(c);
    triangleCount++;
  });
}
}

if (vertexCount === 0) throw new ModelParseError('err.obj.noVertices');
if (triangleCount === 0) {
  throw new ModelParseError('err.obj.noFaces');
}

// Los índices positivos pueden apuntar a vértices que aparecen después: se validan al final.
const positions = new Float32Array(triangleCount * 9);
for (let i = 0; i < triangleCount * 3; i++) {
  const v = faceIndices.data[i] ?? 0;
  if (v >= vertexCount) {
    throw new ModelParseError('err.obj.outOfRange', { index: v + 1, count: vertexCount });
  }
  positions[i * 3] = vertices.data[v * 3] ?? 0;
  positions[i * 3 + 1] = vertices.data[v * 3 + 1] ?? 0;
  positions[i * 3 + 2] = vertices.data[v * 3 + 2] ?? 0;
}
return { positions, triangleCount, format: 'obj', largePolygons };
}

```

## src/stl/threemf.ts · 359 líneas

```
import type { ErrorKey } from '../i18n/es';
import type { Params } from '../i18n/interpolate';
import { ModelParseError } from './errors';
import type { Mesh } from './types';
import { attributes, scanXml } from './xml';
import { readZipDirectory, readZipEntry, zipKey, type ZipEntry } from './zip';

/** Modelo principal cuando `_rels/.rels` no lo indica. */
const DEFAULT_MODEL = '3D/3dmodel.model';
/** Un objeto que se contiene a sí mismo (directa o indirectamente) no termina nunca. */
const MAX_DEPTH = 32;

/**
 * Tope de triángulos de todo el 3MF (contando cada instancia y cada copia). 6 millones son los que
 * caben en un STL de 300 MB (el límite de archivo de la interfaz) y ocupan 216 MB como `Float32Array`;
 * por encima, unas pocas copias de una pieza detallada agotan la memoria y salía un `RangeError` genérico.
 */
export const MAX_3MF_TRIANGLES = 6_000_000;

/** Milímetros que mide cada unidad del 3MF. */
const UNIT_MM: Readonly<Record<string, number>> = {
  micron: 0.001,
  millimeter: 1,
  centimeter: 10,
  inch: 25.4,
  foot: 304.8,
  meter: 1000,
};

/**
 * Transformación afin como en 3MF: 12 números (m00 m01 m02 m10 m11 m12 m20 m21 m22 m30 m31 m32)
 * con convención de vector fila: [x' y' z'] = [x y z] · M3x3 + [m30 m31 m32].
 */
export type Matrix = readonly [number, number, number, number, number, number, number, number, number, number, number, number];

export const IDENTITY: Matrix = [1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0];

/** Aplica primero `a` y después `b` (el producto A·B con vectores fila). */
export function composeMatrix(a: Matrix, b: Matrix): Matrix {
  const at = (r: number, c: number): number => a[r * 3 + c] ?? 0;
  const bt = (r: number, c: number): number => b[r * 3 + c] ?? 0;
  const out: number[] = [];
  for (let r = 0; r < 3; r++) {
    for (let c = 0; c < 3; c++) out.push(at(r, 0) * bt(0, c) + at(r, 1) * bt(1, c) + at(r, 2) * bt(2, c));
  }
  for (let c = 0; c < 3; c++) out.push(at(3, 0) * bt(0, c) + at(3, 1) * bt(1, c) + at(3, 2) * bt(2, c) + bt(3, c));
  return out as unknown as Matrix;
}

/** Signo del determinante de la parte lineal: negativo = la transformación refleja (invierte las caras). */
function determinant(m: Matrix): number {
  const [a, b, c, d, e, f, g, h, i] = m;
  return a * (e * i - f * h) - b * (d * i - f * g) + c * (d * h - e * g);
}

/** Lee el atributo `transform`; vacío o ausente = identidad. */
export function parseTransform(text: string | undefined): Matrix {
  if (text === undefined || text.trim() === '') return IDENTITY;
  const parts = text.trim().split(/\s+/).map(Number);
  if (parts.length !== 12 || parts.some((n) => !Number.isFinite(n))) {
    throw new ModelParseError('err.3mf.transform', { text: text.slice(0, 60) });
  }
  return parts as unknown as Matrix;
}

class Growable<T extends Float32Array | Int32Array> {
  data: T;
  length = 0;

  constructor(private readonly make: (size: number) => T) {
    this.data = make(1024);
  }

  push(value: number): void {
    if (this.length === this.data.length) {

```

```

        const grown = this.make(this.data.length * 2);
        grown.set(this.data);
        this.data = grown;
    }
    this.data[this.length++] = value;
}

done(): T {
    return this.data.slice(0, this.length) as T;
}
}

interface MeshData {
    readonly vertices: Float32Array;
    readonly triangles: Int32Array;
}

interface Component {
    readonly objectId: string;
    readonly path: string | undefined;
    readonly transform: Matrix;
}

interface ModelObject {
    readonly id: string;
    readonly printable: boolean;
    mesh?: MeshData;
    readonly components: Component[];
}

interface BuildItem {
    readonly objectId: string;
    readonly path: string | undefined;
    readonly transform: Matrix;
}

interface ModelFile {
    readonly path: string;
    /** Milímetros por unidad del archivo. */
    readonly unit: number;
    readonly objects: Map<string, ModelObject>;
    readonly build: BuildItem[];
}

function fail(code: ErrorKey, params: Params = {}): never {
    throw new ModelParseError(code, params);
}

/** Recorre el XML de un `.model` y lo deja en objetos con sus mallas, componentes y elementos de la plantilla. */
function parseModelXml(path: string, text: string): ModelFile {
    const objects = new Map<string, ModelObject>();
    const build: BuildItem[] = [];
    let unit = 1;
    let current: ModelObject | null = null;
    let vertices: Growable<Float32Array> | null = null;
    let triangles: Growable<Int32Array> | null = null;
    let inBuild = false;
    let vertexCount = 0;

    scanXml(text, (tag) => {
        if (tag.closing) {
            if (tag.name === 'object' && current) {
                if (vertices && triangles) current.mesh = { vertices: vertices.done(), triangles: triangles.done() };
                current = null;
                vertices = null;
                triangles = null;
            } else if (tag.name === 'build') {
                inBuild = false;
            }
        }
        return;
    })

    switch (tag.name) {
        case 'model': {
            const value = attributes(tag.rawAttributes)['unit'];
            if (value !== undefined) {
                const name = value.trim().toLowerCase();
            }
        }
    }
}

```

```

        if (!Object.hasOwn(UNIT_MM, name)) fail('err.3mf.unit', { unit: value.slice(0, 20) });
        unit = UNIT_MM[name] ?? 1;
    }
    break;
}
case 'object': {
    const a = attributes(tag.rawAttributes);
    const id = a['id'];
    if (id === undefined) fail('err.3mf.objectId', { path });
    const type = (a['type'] ?? 'model').toLowerCase();
    // Soportes, superficies y «otros» no son piezas que se impriman como tal.
    current = { id, printable: type === 'model', components: [] };
    objects.set(current.id, current);
    vertices = null;
    triangles = null;
    vertexCount = 0;
    if (tag.selfClosing) current = null;
    break;
}
case 'mesh':
    if (!current) fail('err.3mf.meshOutside', { path });
    vertices = new Growable((n) => new Float32Array(n));
    triangles = new Growable((n) => new Int32Array(n));
    vertexCount = 0;
    break;
case 'vertex': {
    if (!vertices) fail('err.3mf.vertexOutside', { path });
    const a = attributes(tag.rawAttributes);
    for (const axis of ['x', 'y', 'z'] as const) {
        const value = Number(a[axis]);
        if (a[axis] === undefined || !Number.isFinite(Math.fround(value))) {
            fail('err.3mf.badCoord', { n: vertexCount + 1, path, axis: axis.toUpperCase() });
        }
        vertices?.push(value);
    }
    vertexCount++;
    break;
}
case 'triangle': {
    if (!triangles) fail('err.3mf.triangleOutside', { path });
    const a = attributes(tag.rawAttributes);
    for (const key of ['v1', 'v2', 'v3'] as const) {
        const value = Number(a[key]);
        if (a[key] === undefined || !Number.isInteger(value) || value < 0) {
            fail('err.3mf.badIndex', { path, key });
        }
        triangles?.push(value);
    }
    break;
}
case 'component': {
    if (!current) fail('err.3mf.componentOutside', { path });
    const a = attributes(tag.rawAttributes);
    if (a['objectId'] === undefined) fail('err.3mf.componentId', { path });
    current.components.push({ objectId: a['objectId'], path: a['path'], transform: parseTransform(a['transform']) });
});
    break;
}
case 'build':
    inBuild = !tag.selfClosing;
    break;
case 'item': {
    if (!inBuild) break;
    const a = attributes(tag.rawAttributes);
    if (a['objectId'] === undefined) fail('err.3mf.itemId');
    if (a['printable'] === '0' || a['printable']?.toLowerCase() === 'false') break;
    build.push({ objectId: a['objectId'], path: a['path'], transform: parseTransform(a['transform']) });
    break;
}
default:
    break;
}
});
return { path, unit, objects, build };
}
/** Ruta del modelo principal según `_rels/.rels`, o la de por defecto. */

```

```

function rootModelPath(rels: string | null): string {
  if (rels) {
    let found: string | null = null;
    scanXml(rels, (tag) => {
      if (tag.closing || tag.name !== 'Relationship' || found) return;
      const a = attributes(tag.rawAttributes);
      if ((a['Type'] ?? '').toLowerCase().endsWith('/3dmodel') && a['Target']) found = a['Target'];
    });
    if (found) return found;
  }
  return DEFAULT_MODEL;
}

/** Instancia de una malla con la transformación (en mm) que la lleva a su sitio en la plantilla. */
interface Instance {
  readonly mesh: MeshData;
  readonly matrix: Matrix;
}

function scale(factor: number): Matrix {
  return [factor, 0, 0, 0, factor, 0, 0, 0, factor, 0, 0, 0];
}

/**
 * Lee un 3MF (ZIP con XML): descomprime `3D/3dmodel.model` con `DecompressionStream`, junta
 * todos los objetos de la plantilla de impresión (`build`) aplicando la transformación de cada
 * `item` y de cada `component`, y convierte las unidades a milímetros. Admite componentes que
 * apuntan a otros archivos `.model` del ZIP (extensión de producción, que usan Bambu Studio y
 * PrusaSlicer).
 */
export async function parse3mf(data: ArrayBuffer | Uint8Array): Promise<Mesh> {
  const bytes = data instanceof Uint8Array ? data : new Uint8Array(data);
  if (bytes.byteLength === 0) fail('err.empty');

  const entries = readZipDirectory(bytes);
  const decoder = new TextDecoder('utf-8');
  const read = async (entry: ZipEntry): Promise<string> => decoder.decode(await readZipEntry(bytes, entry));

  const relsEntry = entries.get(zipKey('_rels/.rels'));
  const rootPath = rootModelPath(relsEntry ? await read(relsEntry) : null);
  const rootEntry = entries.get(zipKey(rootPath));
  if (!rootEntry) {
    fail('err.3mf.noModel', { path: rootPath.replace(/^\/, '') });
  }

  const files = new Map<string, ModelFile>();
  const load = async (path: string): Promise<ModelFile> => {
    const key = zipKey(path);
    const cached = files.get(key);
    if (cached) return cached;
    const entry = entries.get(key);
    if (!entry) fail('err.3mf.missingFile', { path: path.replace(/^\/, '') });
    const file = parseModelXml(path, await read(entry));
    files.set(key, file);
    return file;
  };

  const root = await load(rootPath);
  const instances: Instance[] = [];
  let triangles = 0;

  const visit = async (file: ModelFile, objectId: string, matrix: Matrix, depth: number, topLevel: boolean):
  Promise<void> => {
    if (depth > MAX_DEPTH) fail('err.3mf.circular');
    const object = file.objects.get(objectId);
    if (!object) fail('err.3mf.missingObject', { id: objectId });
    if (topLevel && !object.printable) return;

    if (object.mesh) {
      // Del sistema del archivo de la malla al de la plantilla (unidades), luego la matriz, luego a mm.
      const toRoot = composeMatrix(scale(file.unit / root.unit), matrix);
      triangles += object.mesh.triangles.length / 3;
      if (triangles > MAX_3MF_TRIANGLES) fail('err.3mf.tooManyTriangles', { millions: MAX_3MF_TRIANGLES / 1_000_000 });
    };
    instances.push({ mesh: object.mesh, matrix: composeMatrix(toRoot, scale(root.unit)) });
  }
  for (const component of object.components) {

```

```

    const target = component.path ? await load(component.path) : file;
    await visit(target, component.objectId, composeMatrix(component.transform, matrix), depth + 1, false);
  }
};

if (root.build.length > 0) {
  for (const item of root.build) {
    const target = item.path ? await load(item.path) : root;
    await visit(target, item.objectId, item.transform, 0, true);
  }
} else {
  // Sin plantilla (archivo incompleto): se toman los objetos que nadie usa como componente.
  const used = new Set<string>();
  for (const object of root.objects.values()) for (const c of object.components) if (!c.path) used.add(c.objectId);
  for (const object of root.objects.values()) {
    if (!used.has(object.id)) await visit(root, object.id, IDENTITY, 0, true);
  }
}

return meshFromInstances(instances);
}

function meshFromInstances(instances: readonly Instance[]): Mesh {
  let triangleCount = 0;
  for (const { mesh } of instances) triangleCount += mesh.triangles.length / 3;
  if (triangleCount === 0) {
    fail('err.3mf.empty');
  }

  const positions = new Float32Array(triangleCount * 9);
  let out = 0;
  for (const { mesh, matrix } of instances) {
    const { vertices, triangles } = mesh;
    const vertexTotal = vertices.length / 3;
    const [m0, m1, m2, m3, m4, m5, m6, m7, m8, tx, ty, tz] = matrix;
    // Una transformación con determinante negativo refleja la pieza y darle la vuelta a las
    // caras evita que el volumen salga negativo («normales invertidas»).
    const flip = determinant(matrix) < 0;
    for (let t = 0; t < triangles.length; t += 3) {
      const corner = [triangles[t] ?? 0, triangles[t + 1] ?? 0, triangles[t + 2] ?? 0];
      if (flip) corner.reverse();
      for (const v of corner) {
        if (v >= vertexTotal) {
          fail('err.3mf.vertexRange', { index: v + 1, count: vertexTotal });
        }
        const x = vertices[v * 3] ?? 0, y = vertices[v * 3 + 1] ?? 0, z = vertices[v * 3 + 2] ?? 0;
        positions[out++] = x * (m0 ?? 1) + y * (m3 ?? 0) + z * (m6 ?? 0) + (tx ?? 0);
        positions[out++] = x * (m1 ?? 0) + y * (m4 ?? 1) + z * (m7 ?? 0) + (ty ?? 0);
        positions[out++] = x * (m2 ?? 0) + y * (m5 ?? 0) + z * (m8 ?? 1) + (tz ?? 0);
      }
    }
  }
  return { positions, triangleCount, format: '3mf' };
}

```

## src/stl/zip.ts · 161 líneas

```
import { ModelParseError } from './errors';

/** Máximo que se descomprime de una sola entrada (protege de «bombas» ZIP). */
export const MAX_ENTRY_BYTES = 400 * 1024 * 1024;

const SIG_LOCAL = 0x04034b50;
const SIG_CENTRAL = 0x02014b50;
const SIG_END = 0x06054b50;
const END_MIN = 22;
const MAX_COMMENT = 0xffff;

export interface ZipEntry {
  /** Nombre tal como está en el archivo. */
  readonly name: string;
  /** 0 = sin comprimir, 8 = deflate. */
  readonly method: number;
  readonly compressedSize: number;
  readonly size: number;
  /** Posición de los datos comprimidos dentro del archivo. */
  readonly dataOffset: number;
}

/** Clave de búsqueda: sin «/» inicial, con «/» y en minúsculas. */
export function zipKey(name: string): string {
  return name.replace(/\\/g, '/').replace(/^\//, '').toLowerCase();
}

/**
 * Lista las entradas de un ZIP leyendo el directorio central. Si el archivo no lo tiene
 * (está cortado), recorre las cabeceras locales una a una, siempre que estas lleven los
 * tamaños. No admite cifrado ni ZIP64: un modelo de más de 4 GB no cabría en el navegador.
 */
export function readZipDirectory(bytes: Uint8Array): Map<string, ZipEntry> {
  const view = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
  const entries = new Map<string, ZipEntry>();
  const decoder = new TextDecoder('utf-8');

  const endAt = findEnd(view);
  if (endAt < 0) {
    return scanLocalHeaders(bytes, view, decoder);
  }

  const total = view.getUint16(endAt + 10, true);
  const dirSize = view.getUint32(endAt + 12, true);
  let pos = view.getUint32(endAt + 16, true);
  if (total === 0xffff || dirSize === 0xffffffff || pos === 0xffffffff) {
    throw new ModelParseError('err.zip.zip64');
  }

  for (let i = 0; i < total; i++) {
    if (pos + 46 > bytes.length || view.getUint32(pos, true) !== SIG_CENTRAL) {
      throw new ModelParseError('err.zip.directory');
    }
    const flags = view.getUint16(pos + 8, true);
    const method = view.getUint16(pos + 10, true);
    const compressedSize = view.getUint32(pos + 20, true);
    const size = view.getUint32(pos + 24, true);
    const nameLength = view.getUint16(pos + 28, true);
    const extraLength = view.getUint16(pos + 30, true);
    const commentLength = view.getUint16(pos + 32, true);
    const localOffset = view.getUint32(pos + 42, true);
    const name = decoder.decode(bytes.subarray(pos + 46, pos + 46 + nameLength));
    pos += 46 + nameLength + extraLength + commentLength;

    if (flags & 1) throw new ModelParseError('err.zip.encrypted');
    // 0xffffffff = el valor real está en el campo «extra» de ZIP64.
    if (compressedSize === 0xffffffff || size === 0xffffffff || localOffset === 0xffffffff) {
      throw new ModelParseError('err.zip.zip64');
    }
    if (name.endsWith('/')) continue; // carpeta
    if (localOffset + 30 > bytes.length || view.getUint32(localOffset, true) !== SIG_LOCAL) {
      throw new ModelParseError('err.zip.header', { name });
    }
    // Las longitudes de nombre y extra de la cabecera local pueden diferir de las del directorio.
    const dataOffset = localOffset + 30 + view.getUint16(localOffset + 26, true) + view.getUint16(localOffset + 28, true);
  }
}
```

```

    entries.set(zipKey(name), { name, method, compressedSize, size, dataOffset });
  }
  return entries;
}

/** Posición de la firma «fin de directorio central», buscándola desde el final. */
function findEnd(view: DataView): number {
  const last = view.byteLength - END_MIN;
  const first = Math.max(0, last - MAX_COMMENT);
  for (let i = last; i >= first; i--) {
    if (view.getUint32(i, true) === SIG_END) return i;
  }
  return -1;
}

function scanLocalHeaders(bytes: Uint8Array, view: DataView, decoder: TextDecoder): Map<string, ZipEntry> {
  const entries = new Map<string, ZipEntry>();
  let pos = 0;
  while (pos + 30 <= bytes.length && view.getUint32(pos, true) === SIG_LOCAL) {
    const flags = view.getUint16(pos + 6, true);
    const method = view.getUint16(pos + 8, true);
    const compressedSize = view.getUint32(pos + 18, true);
    const size = view.getUint32(pos + 22, true);
    const nameLength = view.getUint16(pos + 26, true);
    const extraLength = view.getUint16(pos + 28, true);
    const name = decoder.decode(bytes.subarray(pos + 30, pos + 30 + nameLength));
    const dataOffset = pos + 30 + nameLength + extraLength;
    if (flags & 1) throw new ModelParseError('err.zip.encrypted');
    if (flags & 8) {
      throw new ModelParseError('err.zip.noDirectory');
    }
    if (dataOffset + compressedSize > bytes.length) {
      throw new ModelParseError('err.zip.truncated');
    }
    if (!name.endsWith('/')) entries.set(zipKey(name), { name, method, compressedSize, size, dataOffset });
    pos = dataOffset + compressedSize;
  }
  if (entries.size === 0) throw new ModelParseError('err.zip.invalid');
  return entries;
}

/** Contenido descomprimido de una entrada. */
export async function readZipEntry(bytes: Uint8Array, entry: ZipEntry, maxBytes = MAX_ENTRY_BYTES):
  Promise<Uint8Array> {
  if (entry.size > maxBytes) {
    throw new ModelParseError('err.zip.tooBig', { name: entry.name, mb: Math.round(entry.size / 1048576) });
  }
  if (entry.dataOffset + entry.compressedSize > bytes.length) {
    throw new ModelParseError('err.zip.truncated');
  }
  const raw = bytes.subarray(entry.dataOffset, entry.dataOffset + entry.compressedSize);
  if (entry.method === 0) return raw;
  if (entry.method !== 8) {
    throw new ModelParseError('err.zip.method', { method: entry.method });
  }
  if (typeof DecompressionStream === 'undefined') {
    throw new ModelParseError('err.zip.noStreams');
  }

  try {
    const stream = new Blob([raw as BlobPart]).stream().pipeThrough(new DecompressionStream('deflate-raw'));
    const reader = stream.getReader();
    const chunks: Uint8Array[] = [];
    let total = 0;
    for (;;) {
      const { done, value } = await reader.read();
      if (done) break;
      total += value.length;
      if (total > maxBytes) {
        await reader.cancel();
        throw new ModelParseError('err.zip.exceeds', { name: entry.name, mb: Math.round(maxBytes / 1048576) });
      }
      chunks.push(value);
    }
    const out = new Uint8Array(total);
    let offset = 0;
    for (const chunk of chunks) {

```

```
        out.set(chunk, offset);
        offset += chunk.length;
    }
    return out;
} catch (error) {
    if (error instanceof ModelParseError) throw error;
    throw new ModelParseError('err.zip.inflate', { name: entry.name });
}
}
```

## src/stl/geometry.ts · 227 líneas

```
import type { BoundingBox, Mesh, MeshStats, Vec3 } from './types';

/** Tamaño de cama por defecto (mm): ancho X, fondo Y, alto Z. */
export const DEFAULT_BED: Vec3 = { x: 220, y: 220, z: 250 };

/** Umbral relativo por debajo del cual el volumen se considera = 0. */
const RELATIVE_VOLUME_EPSILON = 1e-6;

export function computeBounds(positions: Float32Array): BoundingBox {
  if (positions.length === 0) {
    const zero = { x: 0, y: 0, z: 0 };
    return { min: zero, max: zero, size: zero };
  }
  let minX = Infinity, minY = Infinity, minZ = Infinity;
  let maxX = -Infinity, maxY = -Infinity, maxZ = -Infinity;
  for (let i = 0; i < positions.length; i += 3) {
    const x = positions[i] ?? 0, y = positions[i + 1] ?? 0, z = positions[i + 2] ?? 0;
    if (x < minX) minX = x;
    if (y < minY) minY = y;
    if (z < minZ) minZ = z;
    if (x > maxX) maxX = x;
    if (y > maxY) maxY = y;
    if (z > maxZ) maxZ = z;
  }
  return {
    min: { x: minX, y: minY, z: minZ },
    max: { x: maxX, y: maxY, z: maxZ },
    size: { x: maxX - minX, y: maxY - minY, z: maxZ - minZ },
  };
}

/**
 * Calcula volumen, área, caja envolvente y aristas abiertas.
 *
 * Volumen: suma de los volúmenes con signo de los tetraedros que forma cada
 * triángulo con un punto de referencia ( $v1 \cdot (v2 \times v3) / 6$ ). Se usa el centro de
 * la caja como referencia para no perder precisión con piezas lejos del origen.
 */
export function computeStats(mesh: Mesh): MeshStats {
  const p = mesh.positions;
  const bounds = computeBounds(p);
  const cx = (bounds.min.x + bounds.max.x) / 2;
  const cy = (bounds.min.y + bounds.max.y) / 2;
  const cz = (bounds.min.z + bounds.max.z) / 2;

  let signedVolume6 = 0;
  let area2 = 0;

  for (let i = 0; i + 8 < p.length; i += 9) {
    const ax = (p[i] ?? 0) - cx, ay = (p[i + 1] ?? 0) - cy, az = (p[i + 2] ?? 0) - cz;
    const bx = (p[i + 3] ?? 0) - cx, by = (p[i + 4] ?? 0) - cy, bz = (p[i + 5] ?? 0) - cz;
    const qx = (p[i + 6] ?? 0) - cx, qy = (p[i + 7] ?? 0) - cy, qz = (p[i + 8] ?? 0) - cz;

    // Volumen con signo:  $a \cdot (b \times c)$ 
    signedVolume6 += ax * (by * qz - bz * qy) + ay * (bz * qx - bx * qz) + az * (bx * qy - by * qx);

    // Área:  $|(b - a) \times (c - a)| / 2$ 
    const ux = bx - ax, uy = by - ay, uz = bz - az;
    const vx = qx - ax, vy = qy - ay, vz = qz - az;
    const nx = uy * vz - uz * vy, ny = uz * vx - ux * vz, nz = ux * vy - uy * vx;
    area2 += Math.sqrt(nx * nx + ny * ny + nz * nz);
  }

  const signedVolume = signedVolume6 / 6;
  return {
    triangleCount: mesh.triangleCount,
    signedVolume,
    volume: Math.abs(signedVolume),
    surfaceArea: area2 / 2,
    bounds,
    openEdges: countOpenEdges(p),
  };
}

/** Distancia mínima (mm, en cada eje) por debajo de la cual dos vértices se consideran el mismo. */
export const WELD_TOLERANCE = 1e-4;
```

```

/**
 * Lejos del origen el float de 32 bits del STL pierde resolución (a 1000 mm un paso
 * mide ~6e-5 mm), así que la tolerancia crece con el tamaño de las coordenadas.
 */
const WELD_RELATIVE_TOLERANCE = 1e-6;
/** Las celdas de la rejilla miden 16 tolerancias: casi siempre basta con mirar una. */
const WELD_CELL_FACTOR = 16;

/**
 * Suelda vértices casi coincidentes y devuelve un id por vértice (3 números por vértice).
 *
 * Los exportadores no siempre escriben coordenadas idénticas para un vértice compartido
 * (redondeos, conversiones de unidades). Se usa una rejilla espacial; un vértice a
 * menos de `tol` del borde de su celda busca también en la vecina, así dos puntos casi
 * iguales a ambos lados de un borde se unen igualmente.
 */
export function weldVertices(positions: Float32Array, tolerance = WELD_TOLERANCE): { ids: Uint32Array; count: number } {
  const vertexCount = Math.floor(positions.length / 3);
  let maxAbs = 0;
  for (let i = 0; i < vertexCount * 3; i++) {
    const v = Math.abs(positions[i] ?? 0);
    if (v > maxAbs) maxAbs = v;
  }
  const tol = Math.max(tolerance, maxAbs * WELD_RELATIVE_TOLERANCE);
  const cell = tol * WELD_CELL_FACTOR;

  const ids = new Uint32Array(vertexCount);
  // Coordenadas de cada vértice soldado y lista enlazada de los que comparten celda.
  const rep = new Float32Array(vertexCount * 3);
  const nextInCell = new Int32Array(vertexCount);
  const firstInCell = new Map<number, number>();
  const hash = (cx: number, cy: number, cz: number): number =>
    Math.imul(cx, 73856093) ^ Math.imul(cy, 19349663) ^ Math.imul(cz, 83492791);

  let count = 0;
  for (let v = 0; v < vertexCount; v++) {
    const x = positions[v * 3] ?? 0, y = positions[v * 3 + 1] ?? 0, z = positions[v * 3 + 2] ?? 0;
    const cx = Math.floor(x / cell), cy = Math.floor(y / cell), cz = Math.floor(z / cell);
    // Celdas vecinas solo en los ejes en los que el vértice está cerca del borde.
    const x0 = x - cx * cell <= tol ? -1 : 0, x1 = (cx + 1) * cell - x <= tol ? 1 : 0;
    const y0 = y - cy * cell <= tol ? -1 : 0, y1 = (cy + 1) * cell - y <= tol ? 1 : 0;
    const z0 = z - cz * cell <= tol ? -1 : 0, z1 = (cz + 1) * cell - z <= tol ? 1 : 0;

    let found = -1;
    search: for (let dx = x0; dx <= x1; dx++) {
      for (let dy = y0; dy <= y1; dy++) {
        for (let dz = z0; dz <= z1; dz++) {
          // El hash puede colisionar: siempre se comprueban las coordenadas.
          let id = firstInCell.get(hash(cx + dx, cy + dy, cz + dz)) ?? -1;
          while (id >= 0) {
            if (
              Math.abs((rep[id * 3] ?? 0) - x) <= tol &&
              Math.abs((rep[id * 3 + 1] ?? 0) - y) <= tol &&
              Math.abs((rep[id * 3 + 2] ?? 0) - z) <= tol
            ) {
              found = id;
              break search;
            }
            id = nextInCell[id] ?? -1;
          }
        }
      }
    }

    if (found < 0) {
      found = count++;
      rep[found * 3] = x;
      rep[found * 3 + 1] = y;
      rep[found * 3 + 2] = z;
      const key = hash(cx, cy, cz);
      nextInCell[found] = firstInCell.get(key) ?? -1;
      firstInCell.set(key, found);
    }
    ids[v] = found;
  }
  return { ids, count };
}

```

```

}

/**
 * Cuenta las aristas que no comparten exactamente dos triángulos. En una malla
 * cerrada («estanca») el resultado es 0. Antes se sueldan los vértices casi
 * coincidentes (`weldVertices`) para no dar falsos avisos de malla abierta.
 */
export function countOpenEdges(positions: Float32Array, tolerance = WELD_TOLERANCE): number {
  const { ids, count } = weldVertices(positions, tolerance);

  const edges = new Map<number, number>();
  const addEdge = (a: number, b: number): void => {
    if (a === b) return; // arista degenerada (dos vértices soldados en uno)
    const lo = Math.min(a, b), hi = Math.max(a, b);
    const key = lo * count + hi; // exacto: count² < 2⁵³ en cualquier STL que quepa en memoria
    edges.set(key, (edges.get(key) ?? 0) + 1);
  };

  for (let t = 0; t + 2 < ids.length; t += 3) {
    const a = ids[t] ?? 0, b = ids[t + 1] ?? 0, c = ids[t + 2] ?? 0;
    addEdge(a, b);
    addEdge(b, c);
    addEdge(c, a);
  }

  let open = 0;
  for (const count of edges.values()) {
    if (count !== 2) open++;
  }
  return open;
}

export type MeshWarning = 'open-mesh' | 'inverted' | 'too-big' | 'tiny' | 'big-polygons';

/** Por debajo de este tamaño (mm, dimensión mayor) la pieza seguramente se exportó en metros o pulgadas. */
const TINY_PART_MM = 1;

/**
 * Avisos sobre la pieza:
 * - `open-mesh`: volumen ≈ 0 o aristas abiertas: el volumen puede no ser fiable.
 * - `inverted`: volumen con signo negativo (normales hacia dentro).
 * - `too-big`: no cabe en la cama ni girándola 90° sobre Z.
 * - `tiny`: la dimensión mayor mide menos de 1 mm (¿exportada en metros o pulgadas?).
 * - `big-polygons`: el OBJ trae polígonos de más de 200 vértices (`Mesh.largePolygons`), triangulados
 *   en abanico: si son cóncavos, la superficie y el visor salen mal.
 */
export function meshWarnings(stats: MeshStats, bed: Vec3 = DEFAULT_BED, largePolygons = 0): MeshWarning[] {
  const warnings: MeshWarning[] = [];
  const { size } = stats.bounds;
  const boxVolume = size.x * size.y * size.z;
  const nearZero = Math.abs(stats.signedVolume) <= RELATIVE_VOLUME_EPSILON * Math.max(boxVolume, 1);

  if (nearZero || stats.openEdges > 0) {
    warnings.push('open-mesh');
  } else if (stats.signedVolume < 0) {
    warnings.push('inverted');
  }

  if (!fitsBed(size, bed)) {
    warnings.push('too-big');
  }

  if (Math.max(size.x, size.y, size.z) < TINY_PART_MM) {
    warnings.push('tiny');
  }

  if (largePolygons > 0) {
    warnings.push('big-polygons');
  }

  return warnings;
}

/** ¿Cabe la caja en la cama? Se permite girar la pieza 90° sobre el eje Z. */
export function fitsBed(size: Vec3, bed: Vec3): boolean {
  if (size.z > bed.z) return false;
  return (size.x <= bed.x && size.y <= bed.y) || (size.y <= bed.x && size.x <= bed.y);
}

```

### src/stl/analyze.ts · 63 líneas

```
import type { ErrorKey } from '../i18n/es';
import type { Params } from '../i18n/interpolate';
import { ModelParseError } from './errors';
import { computeStats } from './geometry';
import { parseModel } from './model';
import { parseStl } from './parse';
import type { Mesh, MeshStats } from './types';

/** Resultado de leer y medir un modelo (STL, OBJ o 3MF). */
export interface AnalyzedStl {
  readonly mesh: Mesh;
  readonly stats: MeshStats;
}

/** Lee un STL y calcula su geometría (síncrono: el STL no necesita descomprimir nada). */
export function analyzeStl(data: ArrayBuffer | Uint8Array): AnalyzedStl {
  const mesh = parseStl(data);
  return { mesh, stats: computeStats(mesh) };
}

/**
 * Lee un STL, OBJ o 3MF (el formato se deduce del contenido y del nombre) y calcula su
 * geometría. Es lo que hace el worker; también sirve de respaldo sin workers.
 */
export async function analyzeModel(data: ArrayBuffer | Uint8Array, fileName?: string): Promise<AnalyzedStl> {
  const mesh = await parseModel(data, fileName);
  return { mesh, stats: computeStats(mesh) };
}

export interface AnalyzeRequest {
  readonly id: number;
  readonly buffer: ArrayBuffer;
  /** Nombre del archivo: desempata el formato cuando el contenido es texto (OBJ o STL ASCII). */
  readonly fileName?: string;
}

export type AnalyzeResponse =
  | { readonly id: number; readonly ok: true; readonly result: AnalyzedStl }
  | {
    readonly id: number;
    readonly ok: false;
    readonly message: string;
    /** Si es un error de lectura previsto: su código y datos, para traducirlo en la interfaz. */
    readonly parseError: { readonly code: ErrorKey; readonly params: Params } | null;
  };

/**
 * Atiende un mensaje del worker. Devuelve la respuesta y los buffers que se pueden
 * transferir (sin copiarlos) de vuelta al hilo principal.
 */
export async function handleAnalyzeRequest(
  request: AnalyzeRequest,
): Promise<{ response: AnalyzeResponse; transfer: Transferable[] }> {
  try {
    const result = await analyzeModel(request.buffer, request.fileName);
    return { response: { id: request.id, ok: true, result }, transfer: [result.mesh.positions.buffer as ArrayBuffer] };
  } catch (error) {
    const parseError = error instanceof ModelParseError ? { code: error.code, params: error.params } : null;
    const message = error instanceof Error ? error.message : String(error);
    return { response: { id: request.id, ok: false, message, parseError }, transfer: [] };
  }
}
```

## src/stl/analyzer.ts · 91 líneas

```
import { analyzeModel, type AnalyzedStl, type AnalyzeRequest, type AnalyzeResponse } from './analyze';
import { ModelParseError } from './errors';

/** Lo mínimo que se usa de un Worker (permite probarlo con un doble en Node). */
export interface WorkerLike {
  postMessage(message: AnalyzeRequest, transfer: Transferable[]): void;
  addEventListener(type: 'message', listener: (event: MessageEvent<AnalyzeResponse>) => void): void;
  addEventListener(type: 'error', listener: (event: Event) => void): void;
  terminate(): void;
}

function createStlWorker(): WorkerLike {
  // Vite detecta esta forma exacta y empaqueta el worker como un archivo aparte.
  return new Worker(new URL('./stl.worker.ts', import.meta.url), { type: 'module' });
}

interface Pending {
  resolve: (result: AnalyzedStl) => void;
  reject: (error: Error) => void;
}

/**
 * Analiza modelos 3D (STL, OBJ, 3MF) en un Web Worker. Si el navegador no puede crear el worker (o este
 * falla al arrancar), hace el trabajo en el hilo principal: más lento, pero funciona.
 */
export class StlAnalyzer {
  private worker: WorkerLike | null = null;
  private workerFailed = false;
  private nextId = 1;
  private readonly pending = new Map<number, Pending>();

  constructor(private readonly createWorker: () => WorkerLike = createStlWorker) {}

  /** Lee y mide el modelo. El buffer se transfiere al worker: no se debe reutilizar después. */
  analyze(buffer: ArrayBuffer, fileName?: string): Promise<AnalyzedStl> {
    const worker = this.ensureWorker();
    if (!worker) {
      return analyzeModel(buffer, fileName);
    }
    const id = this.nextId++;
    return new Promise<AnalyzedStl>((resolve, reject) => {
      this.pending.set(id, { resolve, reject });
      worker.postMessage(fileName === undefined ? { id, buffer } : { id, buffer, fileName }, [buffer]);
    });
  }

  dispose(): void {
    this.worker?.terminate();
    this.worker = null;
    this.rejectAll(new ModelParseError('err.cancelled'));
  }

  private ensureWorker(): WorkerLike | null {
    if (this.worker || this.workerFailed) return this.worker;
    try {
      const worker = this.createWorker();
      worker.addEventListener('message', (event) => this.onMessage(event.data));
      worker.addEventListener('error', () => {
        // El worker no ha podido cargarse o ha reventado (p. ej. sin memoria): los
        // siguientes archivos se leerán en el hilo principal.
        this.workerFailed = true;
        this.worker?.terminate();
        this.worker = null;
        this.rejectAll(new ModelParseError('err.worker'));
      });
      this.worker = worker;
    } catch {
      this.workerFailed = true;
    }
    return this.worker;
  }

  private onMessage(response: AnalyzeResponse): void {
    const pending = this.pending.get(response.id);
    if (!pending) return;
    this.pending.delete(response.id);
  }
}
```

```

    if (response.ok) {
        pending.resolve(response.result);
    } else {
        pending.reject(
            response.parseError ? new ModelParseError(response.parseError.code, response.parseError.params) : new Error(
response.message),
        );
    }
}

private rejectAll(error: Error): void {
    for (const { reject } of this.pending.values()) reject(error);
    this.pending.clear();
}
}

```

#### src/stl/stl.worker.ts · 16 líneas

```

// Worker que lee el modelo (STL, OBJ o 3MF) y calcula su geometría fuera del hilo principal, para que la
// interfaz no se congele con archivos de cientos de miles de triángulos.
import { handleAnalyzeRequest, type AnalyzeRequest } from './analyze';

// Tipado mínimo del ámbito del worker (la lib «webworker» choca con la lib «dom» del proyecto).
interface WorkerScope {
    addEventListener(type: 'message', listener: (event: MessageEvent<AnalyzeRequest>) => void): void;
    postMessage(message: unknown, transfer: Transferable[]): void;
}

const scope = self as unknown as WorkerScope;

scope.addEventListener('message', (event) => {
    void handleAnalyzeRequest(event.data).then(({ response, transfer }) => scope.postMessage(response, transfer));
});

```

## src/quote/model.ts · 116 líneas

```
import { MATERIALS } from './materials';
import type { QuoteSettings } from './settings';

/** Datos geométricos de la pieza que necesita el presupuesto. */
export interface PartGeometry {
  /** Volumen en mm³. */
  readonly volume: number;
  /** Área de superficie en mm². */
  readonly surfaceArea: number;
}

/** Resultado por copia y total. Volúmenes en mm³, pesos en g, tiempos en h, dinero en €. */
export interface Quote {
  readonly shellVolume: number;
  readonly infillVolume: number;
  /** Volumen impreso por copia. */
  readonly printedVolume: number;
  /** Peso por copia en g. */
  readonly weightGrams: number;
  /** Tiempo estimado por copia en horas (incluye la sobrecarga fija). */
  readonly hoursPerCopy: number;
  readonly copies: number;
  /** Totales para todas las copias. */
  readonly totalWeightGrams: number;
  readonly totalHours: number;
  readonly energyKwh: number;
  readonly materialCost: number;
  readonly energyCost: number;
  /** Coste (material + energía) antes de margen. */
  readonly subtotal: number;
  readonly marginAmount: number;
  readonly total: number;
  readonly totalPerCopy: number;
}

/**
 * Redondea a céntimos. Se pasa antes por 12 cifras significativas para que el ruido
 * de la coma flotante (1,005 guardado como 1,00499999...) no decida el redondeo.
 */
export function roundCents(value: number): number {
  if (!Number.isFinite(value)) return 0;
  return Math.round(Number((value * 100).toFixed(12))) / 100 + 0; // + 0: evita -0
}

/** Grosor de pared en mm: perímetros × ancho de línea. */
export function wallThickness(settings: Pick<QuoteSettings, 'perimeters' | 'lineWidth'>): number {
  return Math.max(0, settings.perimeters) * Math.max(0, settings.lineWidth);
}

/**
 * Volumen impreso por copia (mm³). Modelo simplificado:
 * - cáscara ≈ área × grosor de pared, sin superar el volumen de la pieza;
 * - relleno = relleno% × (volumen - cáscara);
 * - impreso = cáscara + relleno.
 */
export function printedVolume(
  part: PartGeometry,
  settings: Pick<QuoteSettings, 'perimeters' | 'lineWidth' | 'infillPercent'>,
): { shell: number; infill: number; total: number } {
  const volume = Math.max(0, part.volume);
  const shell = Math.min(volume, Math.max(0, part.surfaceArea) * wallThickness(settings));
  const infillRatio = Math.min(100, Math.max(0, settings.infillPercent)) / 100;
  const infill = infillRatio * (volume - shell);
  return { shell, infill, total: shell + infill };
}

/**
 * Presupuesto completo. Función pura: mismos datos, mismo resultado.
 *
 * Los importes se redondean a céntimos línea a línea y el subtotal y el total son la
 * suma de las líneas ya redondeadas, como en una factura: así lo que se ve en el
 * desglose siempre suma el total (antes podía verse 0,08 + 0,00 = 0,09).
 */
export function computeQuote(part: PartGeometry, settings: QuoteSettings): Quote {
  const material = MATERIALS[settings.material];
  const pricePerKg = settings.pricePerKg[settings.material];
```

```

const copies = Math.max(1, Math.round(settings.copies));

const { shell, infill, total: printed } = printedVolume(part, settings);

// mm³ → cm³ (÷1000) × g/cm³ = g
const weightGrams = (printed / 1000) * material.density;

// Tiempo = volumen / caudal + sobrecarga fija (por copia: cada copia es una impresión).
const printSeconds = settings.flowRate > 0 ? printed / settings.flowRate : 0;
const hoursPerCopy = (printSeconds + settings.overheadMinutes * 60) / 3600;

const totalWeightGrams = weightGrams * copies;
const totalHours = hoursPerCopy * copies;
const energyKwh = (settings.powerWatts * totalHours) / 1000;

const materialCost = roundCents((totalWeightGrams / 1000) * pricePerKg);
const energyCost = roundCents(energyKwh * settings.energyPrice);
const subtotal = roundCents(materialCost + energyCost);
const marginAmount = roundCents(subtotal * (settings.marginPercent / 100));
const total = roundCents(subtotal + marginAmount);

return {
  shellVolume: shell,
  infillVolume: infill,
  printedVolume: printed,
  weightGrams,
  hoursPerCopy,
  copies,
  totalWeightGrams,
  totalHours,
  energyKwh,
  materialCost,
  energyCost,
  subtotal,
  marginAmount,
  total,
  totalPerCopy: roundCents(total / copies),
};
}

```

#### src/quote/materials.ts · 24 líneas

```

export type MaterialId = 'PLA' | 'PETG' | 'ABS' | 'TPU';

export interface Material {
  readonly id: MaterialId;
  readonly name: string;
  /** Densidad en g/cm³. */
  readonly density: number;
  /** Precio por defecto en €/kg. */
  readonly defaultPricePerKg: number;
}

export const MATERIALS: Readonly<Record<MaterialId, Material>> = {
  PLA: { id: 'PLA', name: 'PLA', density: 1.24, defaultPricePerKg: 20 },
  PETG: { id: 'PETG', name: 'PETG', density: 1.27, defaultPricePerKg: 24 },
  ABS: { id: 'ABS', name: 'ABS', density: 1.04, defaultPricePerKg: 22 },
  TPU: { id: 'TPU', name: 'TPU', density: 1.21, defaultPricePerKg: 35 },
};

export const MATERIAL_IDS: readonly MaterialId[] = ['PLA', 'PETG', 'ABS', 'TPU'];

export function isMaterialId(value: unknown): value is MaterialId {
  return typeof value === 'string' && (MATERIAL_IDS as readonly string[]).includes(value);
}

```

## src/quote/printers.ts · 147 líneas

```
import { t } from '../i18n';
import type { Key } from '../i18n/es';
import type { QuoteSettings } from './settings';

/** Datos de partida de una impresora. Todos son orientativos (ver `note`). */
export interface PrinterProfile {
  readonly id: string;
  readonly name: string;
  /** Caudal volumétrico medio en mm³/s (uso normal, no el máximo del fabricante). */
  readonly flowRate: number;
  /** Potencia eléctrica media en W durante una impresión típica. */
  readonly powerWatts: number;
  /** Volumen de impresión en mm. */
  readonly bedX: number;
  readonly bedY: number;
  readonly bedZ: number;
  /** De dónde salen los valores y cuánto fiarse de ellos: claves de texto que se unen en una nota (ver `printerNote`). */
  readonly note: readonly Key[];
}

/** Identificador del perfil «Personalizada»: los valores los pone la persona usuaria. */
export const CUSTOM_PRINTER = 'custom';

const BED: Key = 'printer.note.bed';
const ESTIMATE: Key = 'printer.note.estimate';

/**
 * Perfiles precargados. Los valores de cama vienen de las fichas técnicas de cada
 * fabricante; caudal y potencia son estimaciones redondeadas (bastante por debajo del
 * máximo publicitario) y por eso la interfaz los rotula como orientativos.
 */
export const PRINTERS: readonly PrinterProfile[] = [
  {
    id: 'bambu-a1',
    name: 'Bambu Lab A1',
    flowRate: 12,
    powerWatts: 100,
    bedX: 256,
    bedY: 256,
    bedZ: 256,
    note: [BED, ESTIMATE],
  },
  {
    id: 'bambu-p1s',
    name: 'Bambu Lab P1S',
    flowRate: 15,
    powerWatts: 110,
    bedX: 256,
    bedY: 256,
    bedZ: 256,
    note: [BED, 'printer.note.p1s', ESTIMATE],
  },
  {
    id: 'prusa-mk4',
    name: 'Prusa MK4',
    flowRate: 11,
    powerWatts: 100,
    bedX: 250,
    bedY: 210,
    bedZ: 220,
    note: [BED, ESTIMATE],
  },
  {
    id: 'prusa-mini',
    name: 'Prusa MINI+',
    flowRate: 8,
    powerWatts: 70,
    bedX: 180,
    bedY: 180,
    bedZ: 180,
    note: [BED, ESTIMATE],
  },
  {
    id: 'creality-ender3-v3',
    name: 'Crealty Ender-3 V3',
  }
]
```

```

    flowRate: 10,
    powerWatts: 120,
    bedX: 220,
    bedY: 220,
    bedZ: 250,
    note: [BED, 'printer.note.ender3v3', ESTIMATE],
  },
  {
    id: 'creality-k1',
    name: 'Crealitiy K1',
    flowRate: 18,
    powerWatts: 130,
    bedX: 220,
    bedY: 220,
    bedZ: 250,
    note: [BED, 'printer.note.k1', ESTIMATE],
  },
  {
    id: 'elegoo-neptune4',
    name: 'Elegoo Neptune 4',
    flowRate: 12,
    powerWatts: 110,
    bedX: 225,
    bedY: 225,
    bedZ: 265,
    note: [BED, ESTIMATE],
  },
];

/** Nota de origen de un perfil en el idioma activo. */
export function printerNote(printer: PrinterProfile): string {
  return printer.note.map((key) => t(key)).join(' ');
}

const BY_ID: ReadonlyMap<string, PrinterProfile> = new Map(PRINTERS.map((printer) => [printer.id, printer]));

export function getPrinter(id: string): PrinterProfile | undefined {
  return BY_ID.get(id);
}

/** ¿Es un identificador de perfil conocido? «Personalizada» no cuenta: no tiene datos. */
export function isPrinterId(value: unknown): value is string {
  return typeof value === 'string' && BY_ID.has(value);
}

/** Campos de los ajustes que rellena un perfil; editar cualquiera pasa a «Personalizada». */
export const PRINTER_FIELDS = ['flowRate', 'powerWatts', 'bedX', 'bedY', 'bedZ'] as const;
export type PrinterField = (typeof PRINTER_FIELDS)[number];

/**
 * Ajustes con el perfil aplicado: copia caudal, potencia y cama. Con «Personalizada» o un
 * identificador desconocido no cambia ningún valor (solo el identificador).
 */
export function applyPrinter(settings: QuoteSettings, id: string): QuoteSettings {
  const printer = getPrinter(id);
  if (!printer) return { ...settings, printerId: CUSTOM_PRINTER };
  return {
    ...settings,
    printerId: printer.id,
    flowRate: printer.flowRate,
    powerWatts: printer.powerWatts,
    bedX: printer.bedX,
    bedY: printer.bedY,
    bedZ: printer.bedZ,
  };
}

/** ¿Los valores de los ajustes coinciden con los del perfil? */
export function matchesPrinter(settings: QuoteSettings, printer: PrinterProfile): boolean {
  return PRINTER_FIELDS.every((field) => settings[field] === printer[field]);
}

```

### src/quote/tax.ts · 26 líneas

```
import { roundCents } from './model';

/** IVA general en España; es el valor por defecto y se puede cambiar en «Datos del negocio». */
export const DEFAULT_VAT_PERCENT = 21;

export interface TaxedTotal {
  /** Base imponible: el total del presupuesto sin impuestos. */
  readonly base: number;
  readonly vatPercent: number;
  /** Cuota de IVA, redondeada a céntimos. */
  readonly vat: number;
  /** Base + IVA. */
  readonly total: number;
}

/**
 * Aplica el IVA a la base imponible. Como en una factura, la cuota se redondea a
 * céntimos y el total es la suma de las dos cifras ya redondeadas: lo que se ve suma.
 */
export function computeTax(base: number, vatPercent: number): TaxedTotal {
  const rate = Number.isFinite(vatPercent) ? Math.max(0, vatPercent) : 0;
  const roundedBase = roundCents(base);
  const vat = roundCents(roundedBase * (rate / 100));
  return { base: roundedBase, vatPercent: rate, vat, total: roundCents(roundedBase + vat) };
}
```

```
import { DEFAULT_VAT_PERCENT } from './tax';

/** Datos del negocio que salen en el PDF. Se guardan solo en este navegador. */
export interface BusinessProfile {
  readonly name: string;
  /** NIF / CIF u otro identificador fiscal. */
  readonly taxId: string;
  /** Dirección; admite varias líneas. */
  readonly address: string;
  readonly phone: string;
  readonly email: string;
  readonly web: string;
  /** Logotipo como «data URL» PNG o JPEG (ya reducido), o null. Nunca sale del navegador. */
  readonly logo: string | null;
  /** Número del próximo presupuesto; sube solo tras descargar cada PDF. */
  readonly quoteNumber: string;
  /** Días de validez del presupuesto. */
  readonly validityDays: number;
  readonly vatPercent: number;
}

/** Longitudes máximas de los textos (el PDF es de una sola página). */
export const BUSINESS_MAX = {
  name: 80,
  taxId: 30,
  address: 200,
  phone: 30,
  email: 80,
  web: 80,
  quoteNumber: 30,
} as const;

export const BUSINESS_LIMITS = {
  validityDays: { min: 1, max: 365 },
  vatPercent: { min: 0, max: 100 },
} as const;

/** El logotipo guardado no puede pesar más que esto (localStorage tiene ~5 MB en total). */
export const MAX_LOGO_DATA_URL_CHARS = 600_000;

export const DEFAULT_VALIDITY_DAYS = 30;

/** Número de presupuesto inicial: «AAAA-001». */
export function firstQuoteNumber(date: Date = new Date()): string {
  return `${date.getFullYear()}-001`;
}

export const DEFAULT_BUSINESS: BusinessProfile = {
  name: '',
  taxId: '',
  address: '',
  phone: '',
  email: '',
  web: '',
  logo: null,
  quoteNumber: firstQuoteNumber(),
  validityDays: DEFAULT_VALIDITY_DAYS,
  vatPercent: DEFAULT_VAT_PERCENT,
};

const LOGO_DATA_URL = /^data:image\/(png|jpeg);base64,[A-Za-z0-9+/]+=*$/;

/** ¿Es un logotipo aceptable para guardar y para el PDF (PNG o JPEG en base64 y de tamaño razonable)? */
export function isValidLogo(value: unknown): value is string {
  return typeof value === 'string' && value.length <= MAX_LOGO_DATA_URL_CHARS && LOGO_DATA_URL.test(value);
}

/** Quita caracteres de control (salvo saltos de línea si `multiline`), recorta y limita la longitud. */
function cleanText(value: unknown, max: number, multiline = false): string {
  if (typeof value !== 'string') return '';
  // eslint-disable-next-line no-control-regex
  const control = multiline ? /\u0000-\u0009\u000b-\u001f\u007f/g : /\u0000-\u001f\u007f/g;
  return value.replace(/\r\n?/g, '\n').replace(control, ' ').trim().slice(0, max);
}

function cleanNumber(value: unknown, fallback: number, min: number, max: number): number {

```

```

    return typeof value === 'number' && Number.isFinite(value) ? Math.min(max, Math.max(min, value)) : fallback;
}

/** Convierte cualquier valor (p. ej. lo leído de localStorage) en datos de negocio válidos. */
export function normalizeBusiness(input: unknown): BusinessProfile {
  const source = (typeof input === 'object' && input !== null ? input : {}) as Record<string, unknown>;
  const quoteNumber = cleanText(source.quoteNumber, BUSINESS_MAX.quoteNumber);
  return {
    name: cleanText(source.name, BUSINESS_MAX.name),
    taxId: cleanText(source.taxId, BUSINESS_MAX.taxId),
    address: cleanText(source.address, BUSINESS_MAX.address, true),
    phone: cleanText(source.phone, BUSINESS_MAX.phone),
    email: cleanText(source.email, BUSINESS_MAX.email),
    web: cleanText(source.web, BUSINESS_MAX.web),
    logo: isValidLogo(source.logo) ? source.logo : null,
    quoteNumber: quoteNumber === '' ? DEFAULT_BUSINESS.quoteNumber : quoteNumber,
    validityDays: Math.round(
      cleanNumber(source.validityDays, DEFAULT_VALIDITY_DAYS, BUSINESS_LIMITS.validityDays.min, BUSINESS_LIMITS.
        validityDays.max),
    ),
    // Dos decimales como mucho: es lo que se puede escribir y rotular sin que el PDF diga otro tipo que el aplicado.
    vatPercent:
      Math.round(cleanNumber(source.vatPercent, DEFAULT_VAT_PERCENT, BUSINESS_LIMITS.vatPercent.min, BUSINESS_LIMITS.
        vatPercent.max) * 100) / 100,
  };
}

/**
 * Número siguiente: suma 1 al último grupo de cifras y conserva los ceros a la izquierda
 * («2026-009» → «2026-010», «PQ7» → «PQ8»). Sin cifras, añade «-2».
 */
export function nextQuoteNumber(current: string): string {
  const match = /^(.*?)(\d+)(\D*)$/ .exec(current);
  if (!match) return current === '' ? firstQuoteNumber() : `${current}-2`;
  const [, prefix = '', digits = '', suffix = ''] = match;
  const next = String(BigInt(digits) + 1n).padStart(digits.length, '0');
  return `${prefix}${next}${suffix}`;
}

/** Fecha hasta la que vale el presupuesto (`days` días después de `from`). */
export function validUntil(from: Date, days: number): Date {
  const date = new Date(from.getFullYear(), from.getMonth(), from.getDate());
  date.setDate(date.getDate() + Math.max(0, Math.round(days)));
  return date;
}

```

## src/quote/format.ts · 62 líneas

```
import { getLocale } from '../i18n';

/**
 * Valor listo para formatear: lo no finito pasa a 0 y lo que se mostraría como «-0,00»
 * (negativos que redondean a cero) también.
 */
function displayable(value: number, decimals: number): number {
  if (!Number.isFinite(value)) return 0;
  return Math.abs(value) < 0.5 * 10 ** -decimals ? 0 : value;
}

const numberFormats = new Map<string, Intl.NumberFormat>();

/** Número con el formato del idioma activo (es-ES: coma decimal; en-GB: punto) y un nº fijo de decimales. */
export function formatNumber(value: number, decimals = 2): string {
  const locale = getLocale();
  const key = `${locale}:${decimals}`;
  let format = numberFormats.get(key);
  if (!format) {
    format = new Intl.NumberFormat(locale, {
      minimumFractionDigits: decimals,
      maximumFractionDigits: decimals,
    });
    numberFormats.set(key, format);
  }
  return format.format(displayable(value, decimals));
}

const euroFormats = new Map<string, Intl.NumberFormat>();

/** Importe en euros con el formato del idioma activo, p. ej. «12,34 €» o «€12.34». */
export function formatEuro(value: number): string {
  const locale = getLocale();
  let format = euroFormats.get(locale);
  if (!format) {
    format = new Intl.NumberFormat(locale, { style: 'currency', currency: 'EUR' });
    euroFormats.set(locale, format);
  }
  return format.format(displayable(value, 2));
}

/** Duración legible a partir de horas: «45 min», «2 h 05 min». */
export function formatDuration(hours: number): string {
  if (!Number.isFinite(hours) || hours <= 0) return '0 min';
  const totalMinutes = Math.round(hours * 60);
  if (totalMinutes < 1) return '< 1 min';
  const h = Math.floor(totalMinutes / 60);
  const m = totalMinutes % 60;
  if (h === 0) return `${m} min`;
  return `${formatNumber(h, 0)} h ${String(m).padStart(2, '0')} min`;
}

/**
 * Lee un número escrito por una persona: acepta coma o punto decimal y espacios.
 * No acepta separadores de miles. Devuelve NaN si no es un número.
 */
export function parseDecimal(text: string): number {
  const cleaned = text.trim().replace(/\\s+/g, '').replace(',', '.');
  if (!/^\\[-+]?\\d+\\.?\\d*|\\.\\d+$/i.test(cleaned)) return Number.NaN;
  return Number(cleaned);
}
```

## src/pdf/render.ts · 321 líneas

```
// Este módulo importa pdf-lib y fontkit (~1,1 MB sin comprimir, ~500 kB gzip): la interfaz lo carga con import()
// solo cuando se pide un PDF, y `loadFonts` descarga entonces las fuentes (224 kB). Un PDF típico pesa ~30 kB.
import fontkit from '@pdf-lib/fontkit';
import { PDFDocument, rgb, type PDFFont, type PDFImage, type PDFPage } from 'pdf-lib';
import type { PdfRow, QuoteDocument } from './document';
import { loadFonts, type FontBytes } from './fonts';

/** A4 en puntos. */
const PAGE_W = 595.28;
const PAGE_H = 841.89;
const MARGIN = 42;
const CONTENT_W = PAGE_W - 2 * MARGIN;
const COLUMN_W = 240;
const RIGHT_W = 232;
const COLUMN_GAP = CONTENT_W - COLUMN_W - RIGHT_W;
/** Líneas del emisor (NIF, dirección, contacto) que caben sobre la vista 3D. */
const MAX_ISSUER_LINES = 10;

// «Hoja técnica suiza» sobre papel blanco: tinta, gris, línea fina y un único acento naranja.
const INK = rgb(0.067, 0.067, 0.067);
const MUTED = rgb(0.361, 0.349, 0.322);
const LINE = rgb(0.851, 0.839, 0.812);
const ACCENT = rgb(1, 0.353, 0.122);

interface Fonts {
  readonly sans: PDFFont;
  readonly bold: PDFFont;
  readonly mono: PDFFont;
  readonly monoBold: PDFFont;
}

/** Sin anchura o de formato (unión de emojis, selector de variante...): no se dibujan. */
const INVISIBLE = /[\p{Cf}\p{Variation_Selector}]/gu;

/**
 * Las fuentes incrustadas cubren latino, griego y cirílico. Lo que falta (vietnamita, chino, emojis...)
 * se transcribe sin marcas (ã → a) y, si aun así no está, sale como «?».
 */
class Sanitizer {
  private readonly sets = new Map<PDFFont, Set<number>>();

  clean(text: string, font: PDFFont): string {
    let set = this.sets.get(font);
    if (!set) {
      set = new Set(font.getCharacterSet());
      this.sets.set(font, set);
    }
    const has = (char: string): boolean => set.has(char.codePointAt(0) ?? -1);
    let out = '';
    for (const char of text.normalize('NFC').replace(INVISIBLE, '').replace(/\s+/g, ' ')) {
      if (has(char)) {
        out += char;
        continue;
      }
      // Guiones raros (p. ej. el que no permite salto de línea) → «-»; el resto, sin marcas diacríticas.
      const plain = /\p{Pd}/u.test(char) ? '-' : char.normalize('NFD').replace(/\p{M}/gu, '');
      out += plain !== '' && [...plain].every(has) ? plain : '?';
    }
    return out;
  }
}

/** Bytes de un PDF de presupuesto de una página. */
export async function renderQuotePdf(doc: QuoteDocument, fontBytes?: FontBytes): Promise<Uint8Array> {
  const bytes = fontBytes ?? (await loadFonts());
  const pdf = await PDFDocument.create();
  pdf.registerFontkit(fontkit);
  pdf.setTitle(doc.metadata.title);
  pdf.setAuthor(doc.metadata.author);
  pdf.setCreator('printquote');
  pdf.setProducer('printquote');
  pdf.setCreationDate(doc.date);
  pdf.setModificationDate(doc.date);
  pdf.setLanguage(doc.metadata.language);

  // subset: solo viajan los glifos usados (un PDF de ~50 kB en vez de ~250 kB).
```

```

const embed = (data: Uint8Array): Promise<PDFFont> => pdf.embedFont(data, { subset: true });
const fonts: Fonts = {
  sans: await embed(bytes.sans),
  bold: await embed(bytes.bold),
  mono: await embed(bytes.mono),
  monoBold: await embed(bytes.monoBold),
};
const sanitizer = new Sanitizer();
const page = pdf.addPage([PAGE_W, PAGE_H]);

// Imágenes: si una no se puede leer, el PDF sale igualmente sin ella.
const embedImage = async (dataUrl: string | null): Promise<PDFImage | null> => {
  if (!dataUrl) return null;
  try {
    return dataUrl.startsWith('data:image/jpeg') ? await pdf.embedJpg(dataUrl) : await pdf.embedPng(dataUrl);
  } catch {
    return null;
  }
};
const logo = await embedImage(doc.logo);
const snapshot = await embedImage(doc.image);

const draw = new Drawer(page, fonts, sanitizer);

// — Cabecera: logotipo o marca + nombre, y a la derecha el número —
let top = MARGIN;
const numberText = doc.meta[0]?.[1] ?? '';
const numberSize = Math.min(16, Math.max(9, 190 / (Math.max(numberText.length, 1) * 0.6)));
const rightBlockW = 200;

let nameX = MARGIN;
let headerH = 30;
if (logo) {
  const scaled = fit(logo.width, logo.height, 150, 52);
  page.drawImage(logo, { x: MARGIN, y: PAGE_H - top - scaled.height, width: scaled.width, height: scaled.height });
  nameX = MARGIN + scaled.width + 14;
  headerH = Math.max(headerH, scaled.height);
} else {
  page.drawRectangle({ x: MARGIN, y: PAGE_H - top - 15, width: 12, height: 12, color: ACCENT });
  nameX = MARGIN + 22;
}
const nameLines = draw.wrap(doc.businessName, fonts.bold, 16, CONTENT_W - rightBlockW - (nameX - MARGIN) - 12).
slice(0, 3);
nameLines.forEach((line, i) => draw.text(line, nameX, top + 13 + i * 19, { font: fonts.bold, size: 16 }));
headerH = Math.max(headerH, nameLines.length * 19 + 4);

draw.text(doc.title.toLocaleUpperCase(doc.metadata.language), MARGIN + CONTENT_W, top + 8, {
  font: fonts.bold,
  size: 8,
  color: MUTED,
  align: 'right',
});
draw.text(numberText, MARGIN + CONTENT_W, top + 8 + numberSize + 4, {
  font: fonts.monoBold,
  size: numberSize,
  align: 'right',
  maxWidth: rightBlockW,
});

top += headerH + 14;
draw.rule(top, MARGIN, MARGIN + CONTENT_W, 1, INK);

// — Fecha, validez —
top += 16;
const metaRows = doc.meta.slice(1);
const metaColW = CONTENT_W / Math.max(metaRows.length, 1);
metaRows.forEach(([label, value], i) => {
  const x = MARGIN + i * metaColW;
  draw.text(label.toLocaleUpperCase(doc.metadata.language), x, top, { font: fonts.sans, size: 7.5, color: MUTED });
  draw.text(value, x, top + 15, { font: fonts.mono, size: 10.5, maxWidth: metaColW - 10 });
});
top += 34;
draw.rule(top, MARGIN, MARGIN + CONTENT_W, 0.5, LINE);
top += 22;

// — Columna izquierda: emisor y vista 3D —
const leftX = MARGIN;

```

```

const rightX = MARGIN + COLUMN_W + COLUMN_GAP;
const rightW = MARGIN + CONTENT_W - rightX;
let left = top;
const issuer = doc.issuerLines.flatMap((line) => draw.wrap(line, fonts.sans, 9.5, COLUMN_W));
// Una dirección interminable no puede empujar la vista 3D fuera de la página.
if (issuer.length > MAX_ISSUER_LINES) {
  issuer.length = MAX_ISSUER_LINES;
  issuer[MAX_ISSUER_LINES - 1] = draw.truncate(`${issuer[MAX_ISSUER_LINES - 1]} ?? '...'`, fonts.sans, 9.5,
COLUMN_W);
}
if (issuer.length > 0) {
  left = draw.title(doc.issuerTitle, leftX, left, COLUMN_W);
  for (const line of issuer) {
    draw.text(line, leftX, left + 9, { font: fonts.sans, size: 9.5 });
    left += 13.5;
  }
}
if (snapshot) {
  if (issuer.length > 0) left += 14;
  const scaled = fit(snapshot.width, snapshot.height, COLUMN_W, 170);
  page.drawImage(snapshot, { x: leftX, y: PAGE_H - left - scaled.height, width: scaled.width, height: scaled.height
});
  page.drawRectangle({
    x: leftX,
    y: PAGE_H - left - scaled.height,
    width: scaled.width,
    height: scaled.height,
    borderColor: LINE,
    borderWidth: 0.5,
  });
  left += scaled.height;
}

// — Columna derecha: pieza y desglose —
let right = top;
right = draw.title(doc.pieceTitle, rightX, right, rightW);
right = draw.table(doc.pieceRows, rightX, right, rightW);
right += 20;
right = draw.title(doc.breakdownTitle, rightX, right, rightW);
right = draw.table(doc.breakdownRows, rightX, right, rightW);

// — Totales y notas —
top = Math.max(left, right) + 22;
const totalsW = rightW; // alineado con la columna derecha
const totalsX = rightX;
let totals = top;
const taxRows = doc.totalRows.slice(0, -1);
totals = draw.table(taxRows, totalsX, totals, totalsW);
draw.rule(totals + 6, totalsX, totalsX + totalsW, 1.5, INK);
totals += 16;
const grand = doc.totalRows[doc.totalRows.length - 1];
if (grand) {
  page.drawRectangle({ x: totalsX, y: PAGE_H - totals - 8, width: 9, height: 9, color: ACCENT });
  draw.text(grand[0].toLocaleUpperCase(doc.metadata.language), totalsX + 16, totals + 8, { font: fonts.bold, size:
8.5 });
  draw.text(grand[1], totalsX + totalsW, totals + 32, { font: fonts.monoBold, size: 22, align: 'right', maxWidth:
totalsW });
}

let notes = top + 4;
for (const note of doc.notes) {
  for (const line of draw.wrap(note, fonts.sans, 8, COLUMN_W)) {
    draw.text(line, MARGIN, notes + 7, { font: fonts.sans, size: 8, color: MUTED });
    notes += 11;
  }
  notes += 5;
}

// — Pie —
const footerTop = PAGE_H - MARGIN + 6;
draw.rule(footerTop - 10, MARGIN, MARGIN + CONTENT_W, 0.5, LINE);
draw.text(doc.footer, MARGIN, footerTop + 2, { font: fonts.sans, size: 7.5, color: MUTED });

return pdf.save();
}

/** Escala `w x h` para que quepa en `maxW x maxH` sin deformarse ni ampliarse. */

```

```

export function fit(w: number, h: number, maxW: number, maxH: number): { width: number; height: number } {
  const ratio = Math.min(maxW / w, maxH / h, 1);
  return { width: w * ratio, height: h * ratio };
}

interface TextOptions {
  readonly font: PDFFont;
  readonly size: number;
  readonly color?: ReturnType<typeof rgb>;
  readonly align?: 'left' | 'right';
  /** Si el texto es más ancho, se acorta con «...». */
  readonly maxWidth?: number;
}

/** Dibuja con coordenadas «desde arriba» (el origen de PDF es la esquina inferior izquierda). */
class Drawer {
  constructor(
    private readonly page: PDFPage,
    private readonly fonts: Fonts,
    private readonly sanitizer: Sanitizer,
  ) {}

  /** `top` es la línea base del texto medida desde el borde superior de la página. */
  text(value: string, x: number, top: number, options: TextOptions): void {
    const { font, size, color = INK, align = 'left', maxWidth } = options;
    let text = this.sanitizer.clean(value, font);
    if (maxWidth !== undefined) text = this.truncate(text, font, size, maxWidth);
    const width = font.widthOfTextAtSize(text, size);
    this.page.drawText(text, {
      x: align === 'right' ? x - width : x,
      y: PAGE_H - top,
      size,
      font,
      color,
    });
  }

  rule(top: number, x1: number, x2: number, thickness: number, color: ReturnType<typeof rgb>): void {
    this.page.drawLine({ start: { x: x1, y: PAGE_H - top }, end: { x: x2, y: PAGE_H - top }, thickness, color });
  }

  /** Título de sección (mayúsculas pequeñas) con filete; devuelve la siguiente posición. */
  title(label: string, x: number, top: number, width: number): number {
    this.text(label.toLocaleUpperCase(), x, top + 8, { font: this.fonts.bold, size: 8, color: MUTED });
    this.rule(top + 14, x, x + width, 0.75, INK);
    return top + 14;
  }

  /** Tabla «etiqueta ... valor»: etiqueta en sans, valor en monoespaciada alineado a la derecha. */
  table(rows: readonly PdfRow[], x: number, top: number, width: number): number {
    let y = top;
    for (const [label, value] of rows) {
      const valueFont = this.fonts.mono;
      const valueWidth = valueFont.widthOfTextAtSize(this.sanitizer.clean(value, valueFont), 9.5);
      const labelMax = Math.max(40, width - valueWidth - 12);
      this.text(label, x, y + 13, { font: this.fonts.sans, size: 9, color: MUTED, maxWidth: labelMax });
      this.text(value, x + width, y + 13, { font: valueFont, size: 9.5, align: 'right', maxWidth: width - 40 });
      y += 18;
      this.rule(y, x, x + width, 0.5, LINE);
    }
    return y;
  }

  truncate(text: string, font: PDFFont, size: number, maxWidth: number): string {
    if (font.widthOfTextAtSize(text, size) <= maxWidth) return text;
    let end = text.length;
    while (end > 1 && font.widthOfTextAtSize(`${text.slice(0, end)}...`, size) > maxWidth) end--;
    return `${text.slice(0, end)}...`;
  }

  /** Parte el texto en líneas que caben en `maxWidth`; las palabras más largas se cortan. */
  wrap(value: string, font: PDFFont, size: number, maxWidth: number): string[] {
    const text = this.sanitizer.clean(value, font);
    const lines: string[] = [];
    let current = '';
    const fits = (candidate: string): boolean => font.widthOfTextAtSize(candidate, size) <= maxWidth;
    for (const word of text.split(' ')) {

```

```

    const candidate = current === '' ? word : `${current} ${word}`;
    if (fits(candidate)) {
        current = candidate;
        continue;
    }
    if (current !== '') lines.push(current);
    current = word;
    while (!fits(current) && current.length > 1) {
        let cut = current.length - 1;
        while (cut > 1 && !fits(current.slice(0, cut))) cut--;
        lines.push(current.slice(0, cut));
        current = current.slice(cut);
    }
    if (current !== '') lines.push(current);
    return lines.length > 0 ? lines : [''];
}
}

```

## src/pdf/fonts.ts · 51 líneas

```

/**
 * Fuentes del PDF (Noto Sans y JetBrains Mono, licencia OFL; ver `fonts/LEEME.txt`). Son archivos
 * aparte: Vite los emite como recursos y solo se piden al pulsar «Descargar PDF».
 */
export interface FontBytes {
    readonly sans: Uint8Array;
    readonly bold: Uint8Array;
    readonly mono: Uint8Array;
    readonly monoBold: Uint8Array;
}

/** No se pudieron descargar las fuentes (sin conexión, 404...). La interfaz lo distingue por `name`. */
export class FontLoadError extends Error {
    override readonly name = 'FontLoadError';
}

async function get(url: URL): Promise<Uint8Array> {
    const response = await fetch(url);
    if (!response.ok) throw new Error(`HTTP ${response.status}`);
    return new Uint8Array(await response.arrayBuffer());
}

let loading: Promise<FontBytes> | null = null;

/** Descarga las fuentes una sola vez por sesión; si falla, se olvida para poder reintentar. */
export function loadFonts(): Promise<FontBytes> {
    loading ??= download().catch((error: unknown) => {
        loading = null;
        throw error;
    });
    return loading;
}

async function download(): Promise<FontBytes> {
    try {
        return await fetchAll();
    } catch (error) {
        throw new FontLoadError(`No se han podido descargar las fuentes del PDF: ${String(error)}`, { cause: error });
    }
}

async function fetchAll(): Promise<FontBytes> {
    const [sans, bold, mono, monoBold] = await Promise.all([
        get(new URL('./fonts/NotoSans-Regular.subset.ttf', import.meta.url)),
        get(new URL('./fonts/NotoSans-Bold.subset.ttf', import.meta.url)),
        get(new URL('./fonts/JetBrainsMono-Regular.subset.ttf', import.meta.url)),
        get(new URL('./fonts/JetBrainsMono-Bold.subset.ttf', import.meta.url)),
    ]);
    return { sans, bold, mono, monoBold };
}

```

## src/pdf/document.ts · 149 líneas

```
import { getLang } from '../i18n';
import type { BusinessProfile } from '../quote/business';
import { validUntil } from '../quote/business';
import { formatDuration, formatEuro, formatNumber } from '../quote/format';
import { MATERIALS } from '../quote/materials';
import type { Quote } from '../quote/model';
import type { QuoteSettings } from '../quote/settings';
import { computeTax, type TaxedTotal } from '../quote/tax';
import type { MeshStats } from '../stl/types';
import { pdfLabels } from './labels';

/** Fila «etiqueta – valor» de una tabla del PDF. */
export type PdfRow = readonly [label: string, value: string];

/** Todos los textos fijos del PDF (se traducen junto con la interfaz). */
export interface PdfLabels {
  readonly title: string;
  readonly number: string;
  readonly date: string;
  readonly validUntil: string;
  readonly issuer: string;
  readonly taxId: string;
  readonly piece: string;
  readonly file: string;
  readonly dimensions: string;
  readonly volume: string;
  readonly material: string;
  readonly infill: string;
  readonly copies: string;
  readonly breakdown: string;
  readonly weight: string;
  readonly time: string;
  readonly timeEstimate: string;
  readonly materialCost: string;
  readonly energyCost: string;
  readonly subtotal: string;
  readonly margin: string;
  readonly perCopy: string;
  readonly taxBase: string;
  readonly vat: string;
  readonly totalWithVat: string;
  readonly noBusinessName: string;
  readonly noteEstimate: string;
  readonly noteValidity: string;
  readonly footer: string;
  readonly pdfTitle: string;
  readonly language: string;
  /** Configuración regional para las fechas largas («30 de septiembre de 2026»). */
  readonly dateLocale: string;
}

/** Contenido del PDF ya calculado y formateado; el dibujo se hace en `render.ts`. */
export interface QuoteDocument {
  readonly title: string;
  readonly businessName: string;
  readonly logo: string | null;
  /** N°, fecha y validez. */
  readonly meta: readonly PdfRow[];
  readonly issuerTitle: string;
  /** Líneas del emisor (NIF, dirección, teléfono, correo, web). */
  readonly issuerLines: readonly string[];
  readonly pieceTitle: string;
  readonly pieceRows: readonly PdfRow[];
  readonly image: string | null;
  readonly breakdownTitle: string;
  readonly breakdownRows: readonly PdfRow[];
  /** Base imponible, IVA y total con IVA (la última fila es el total). */
  readonly totalRows: readonly PdfRow[];
  readonly tax: TaxedTotal;
  readonly notes: readonly string[];
  readonly footer: string;
  readonly metadata: { readonly title: string; readonly author: string; readonly language: string };
  readonly date: Date;
}

export interface QuoteDocumentInput {
```

```

readonly business: BusinessProfile;
readonly fileName: string;
readonly stats: MeshStats;
readonly settings: QuoteSettings;
readonly quote: Quote;
/** Vista 3D como «data URL» PNG, o null. */
readonly image: string | null;
readonly date?: Date;
readonly labels?: PdfLabels;
}

/** Calcula y formatea todo lo que lleva el PDF. Función pura: mismos datos, mismo contenido. */
export function buildQuoteDocument(input: QuoteDocumentInput): QuoteDocument {
  const { business, fileName, stats, settings, quote, image } = input;
  const date = input.date ?? new Date();
  const labels = input.labels ?? pdfLabels(getLang());
  const material = MATERIALS[settings.material];
  const { size } = stats.bounds;
  const tax = computeTax(quote.total, business.vatPercent);
  const dateFormat = new Intl.DateTimeFormat(labels.dateLocale, { dateStyle: 'long' });

  const issuerLines: string[] = [];
  if (business.taxId) issuerLines.push(`${labels.taxId}: ${business.taxId}`);
  if (business.address) issuerLines.push(...business.address.split('\n').filter((line) => line.trim() !== ''));
  for (const line of [business.phone, business.email, business.web]) if (line) issuerLines.push(line);

  const breakdownRows: PdfRow[] = [
    [labels.copies, formatNumber(quote.copies, 0)],
    [labels.weight, `${formatNumber(quote.totalWeightGrams, 1)} g`],
    [`${labels.time} (${labels.timeEstimate})`, formatDuration(quote.totalHours)],
    [labels.materialCost, formatEuro(quote.materialCost)],
    [`${labels.energyCost} (${formatNumber(quote.energyKwh, 2)} kwh)`, formatEuro(quote.energyCost)],
    [labels.subtotal, formatEuro(quote.subtotal)],
    [`${labels.margin} (${formatNumber(settings.marginPercent, 0)} %)`, formatEuro(quote.marginAmount)],
  ];
  if (quote.copies > 1) breakdownRows.push([labels.perCopy, formatEuro(quote.totalPerCopy)]);

  const vatDecimals = tax.vatPercent % 1 === 0 ? 0 : Math.round(tax.vatPercent * 10) === tax.vatPercent * 10 ? 1 : 2;
  return {
    title: labels.title,
    businessName: business.name || labels.noBusinessName,
    logo: business.logo,
    meta: [
      [labels.number, business.quoteNumber],
      [labels.date, dateFormat.format(date)],
      [labels.validUntil, dateFormat.format(validUntil(date, business.validityDays))],
    ],
    issuerTitle: labels.issuer,
    issuerLines,
    pieceTitle: labels.piece,
    pieceRows: [
      [labels.file, fileName],
      [labels.dimensions, `${formatNumber(size.x, 1)} × ${formatNumber(size.y, 1)} × ${formatNumber(size.z, 1)} mm`],
      [labels.volume, `${formatNumber(stats.volume / 1000, 2)} cm³`],
      [labels.material, `${material.name} · ${formatEuro(settings.pricePerKg[settings.material])}/kg`],
      [labels.infill, `${formatNumber(settings.infillPercent, 0)} % · ${formatNumber(settings.perimeters, 0)} × ${formatNumber(settings.lineWidth, 2)} mm`],
    ],
    image,
    breakdownTitle: labels.breakdown,
    breakdownRows,
    totalRows: [
      [labels.taxBase, formatEuro(tax.base)],
      [`${labels.vat} (${formatNumber(tax.vatPercent, vatDecimals)} %)`, formatEuro(tax.vat)],
      [labels.totalWithVat, formatEuro(tax.total)],
    ],
    tax,
    notes: [labels.noteEstimate, labels.noteValidity],
    footer: labels.footer,
    metadata: { title: `${labels.pdfTitle} ${business.quoteNumber}`, author: business.name, language: labels.language },
    date,
  };
}

```

## src/viewer/viewer.ts · 256 líneas

```
import {
  AmbientLight,
  BufferAttribute,
  BufferGeometry,
  Color,
  DirectionalLight,
  EdgesGeometry,
  Float32BufferAttribute,
  Group,
  HemisphereLight,
  LineBasicMaterial,
  LineSegments,
  Mesh,
  MeshStandardMaterial,
  PerspectiveCamera,
  Scene,
  Vector3,
  WebGLRenderer,
} from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import type { Vec3 } from '../stl/types';
import { keyZoomFactor } from './zoom';

export interface ViewerTheme {
  background: string;
  ink: string;
  line: string;
  accent: string;
}

/** Dirección desde la que mira la cámara al encuadrar (frente-derecha, algo elevada). */
const VIEW_DIRECTION = new Vector3(0.9, -1.35, 0.85).normalize();
const FOV = 35;

/**
 * Visor 3D de la pieza sobre la cama. Trabaja con Z hacia arriba, como los laminadores.
 * Solo renderiza cuando algo cambia (no hay bucle continuo) para no gastar batería.
 */
export class Viewer {
  private readonly renderer: WebGLRenderer;
  private readonly scene = new Scene();
  private readonly camera = new PerspectiveCamera(FOV, 1, 0.1, 20000);
  private readonly controls: OrbitControls;
  private readonly bedGroup = new Group();
  private readonly partGroup = new Group();
  private readonly partMaterial = new MeshStandardMaterial({ roughness: 0.92, metalness: 0, flatShading: true });
  private readonly edgeMaterial = new LineBasicMaterial({ transparent: true, opacity: 0.35 });
  private readonly minorMaterial = new LineBasicMaterial();
  private readonly majorMaterial = new LineBasicMaterial();
  private readonly outlineMaterial = new LineBasicMaterial();
  private readonly resizeObserver: ResizeObserver;
  private bed: Vec3 = { x: 220, y: 220, z: 250 };
  private partSize: Vec3 | null = null;
  private frame = 0;

  constructor(private readonly container: HTMLElement, theme: ViewerTheme) {
    this.renderer = new WebGLRenderer({ antialias: true });
    this.renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
    this.renderer.domElement.setAttribute('aria-hidden', 'true');
    container.appendChild(this.renderer.domElement);

    this.camera.up.set(0, 0, 1);
    this.controls = new OrbitControls(this.camera, this.renderer.domElement);
    // Inercia al soltar, salvo que se haya pedido reducir el movimiento.
    this.controls.enableDamping = !window.matchMedia('(prefers-reduced-motion: reduce)').matches;
    this.controls.dampingFactor = 0.12;
    // Teclado (con el foco en el visor): flechas para desplazar, Mayús + flechas para girar y + / - para acercar.
    this.controls.keyRotateSpeed = 15;
    this.controls.listenToKeyEvents(container);
    container.addEventListener('keydown', this.onKeyDown);
    this.controls.addEventListener('change', () => this.requestRender());

    this.scene.add(new HemisphereLight(0xffffffff, 0x8a8580, 1.6));
    this.scene.add(new AmbientLight(0xffffffff, 0.35));
    const key = new DirectionalLight(0xffffffff, 1.6);
    key.position.set(-1.2, -2, 3);
  }

  requestRender() {
    this.frame++;
    this.renderer.render(this.scene, this.camera);
  }
}
```

```

    this.scene.add(key);
    const rim = new DirectionalLight(0xffffffff, 0.5);
    rim.position.set(2, 3, 1);
    this.scene.add(rim);

    this.scene.add(this.bedGroup, this.partGroup);
    this.setTheme(theme);
    this.buildBed();

    this.resizeObserver = new ResizeObserver(() => this.resize());
    this.resizeObserver.observe(container);
    this.resize();
    this.resetCamera();
}

setTheme(theme: ViewerTheme): void {
    this.scene.background = new Color(theme.background);
    this.partMaterial.color.set(theme.accent);
    this.edgeMaterial.color.set(theme.ink);
    this.minorMaterial.color.set(theme.line);
    this.majorMaterial.color.set(new Color(theme.line).lerp(new Color(theme.ink), 0.25));
    this.outlineMaterial.color.set(theme.ink);
    this.requestRender();
}

/** Cambia el tamaño de la cama (mm) y redibuja la rejilla. */
setBed(bed: Vec3): void {
    if (bed.x === this.bed.x && bed.y === this.bed.y && bed.z === this.bed.z) return;
    this.bed = bed;
    this.buildBed();
    this.requestRender();
}

/**
 * Muestra una malla (9 números por triángulo, mm). Se centra en la cama y se
 * apoya sobre ella (Z mínima = 0) solo para verla; los cálculos usan el original.
 */
setPart(positions: Float32Array): void {
    this.clearPart();
    const geometry = new BufferGeometry();
    geometry.setAttribute('position', new BufferAttribute(positions.slice(), 3));
    geometry.computeBoundingBox();
    const box = geometry.boundingBox;
    if (!box) return;
    const center = box.getCenter(new Vector3());
    geometry.translate(-center.x, -center.y, -box.min.z);
    geometry.computeVertexNormals();

    const mesh = new Mesh(geometry, this.partMaterial);
    this.partGroup.add(mesh);

    // Aristas marcadas (ángulo > 30°) para leer bien la forma, como en un plano.
    if (positions.length / 9 <= 400_000) {
        this.partGroup.add(new LineSegments(new EdgesGeometry(geometry, 30), this.edgeMaterial));
    }

    const size = box.getSize(new Vector3());
    this.partSize = { x: size.x, y: size.y, z: size.z };
    this.resetCamera();
}

clearPart(): void {
    for (const child of [...this.partGroup.children]) {
        this.partGroup.remove(child);
        if (child instanceof Mesh || child instanceof LineSegments) child.geometry.dispose();
    }
    this.partSize = null;
    this.requestRender();
}

/** Encuadra la pieza (o la cama si no hay pieza) desde la vista por defecto. */
resetCamera(): void {
    const target = new Vector3();
    let radius: number;
    if (this.partSize) {
        const s = this.partSize;
        target.set(0, 0, s.z / 2);
    }

```

```

        radius = Math.hypot(s.x, s.y, s.z) / 2;
    } else {
        radius = Math.hypot(this.bed.x, this.bed.y) / 2;
    }
    radius = Math.max(radius, 1);

    const vFov = (FOV * Math.PI) / 180;
    const hFov = 2 * Math.atan(Math.tan(vFov / 2) * this.camera.aspect);
    const distance = (radius / Math.sin(Math.min(vFov, hFov) / 2)) * 1.3;

    this.camera.near = Math.max(0.1, distance / 1000);
    this.camera.far = distance * 50;
    this.camera.position.copy(target).addScaledVector(VIEW_DIRECTION, distance);
    this.camera.updateProjectionMatrix();
    this.controls.target.copy(target);
    this.controls.minDistance = radius * 0.2;
    this.controls.maxDistance = distance * 8;
    this.controls.update();
    this.requestRender();
}

/** Imagen PNG de la vista actual (para la hoja impresa). */
snapshot(): string {
    this.renderer.render(this.scene, this.camera);
    return this.renderer.domElement.toDataURL('image/png');
}

dispose(): void {
    cancelAnimationFrame(this.frame);
    this.resizeObserver.disconnect();
    this.container.removeEventListener('keydown', this.onKeyDown);
    this.controls.dispose();
    this.clearPart();
    this.renderer.dispose();
    this.renderer.domElement.remove();
}

/** Zoom con el teclado: mueve la cámara sobre la línea al objetivo; `update()` la limita a min/maxDistance. */
private readonly onKeyDown = (event: KeyboardEvent): void => {
    const factor = keyZoomFactor(event, this.partSize !== null);
    if (factor === null) return;
    event.preventDefault();
    const offset = this.camera.position.clone().sub(this.controls.target);
    this.camera.position.copy(this.controls.target).add(offset.multiplyScalar(factor));
    this.controls.update();
};

private requestRender(): void {
    if (this.frame) return;
    this.frame = requestAnimationFrame(() => {
        this.frame = 0;
        // Con amortiguación, update() devuelve true mientras la cámara siga moviéndose.
        const moving = this.controls.update();
        this.renderer.render(this.scene, this.camera);
        if (moving) this.requestRender();
    });
}

private resize(): void {
    const width = Math.max(1, this.container.clientWidth);
    const height = Math.max(1, this.container.clientHeight);
    this.renderer.setSize(width, height, false);
    this.camera.aspect = width / height;
    this.camera.updateProjectionMatrix();
    this.requestRender();
}

/** Rejilla de la cama: línea fina cada 10 mm, marcada cada 50 mm y contorno. */
private buildBed(): void {
    for (const child of [...this.bedGroup.children]) {
        this.bedGroup.remove(child);
        if (child instanceof LineSegments) child.geometry.dispose();
    }
    const { x: w, y: d } = this.bed;
    const x0 = -w / 2, y0 = -d / 2;
    const minor: number[] = [];
    const major: number[] = [];

```

```

    for (let x = 10; x < w; x += 10) {
      (x % 50 === 0 ? major : minor).push(x0 + x, y0, 0, x0 + x, y0 + d, 0);
    }
    for (let y = 10; y < d; y += 10) {
      (y % 50 === 0 ? major : minor).push(x0, y0 + y, 0, x0 + w, y0 + y, 0);
    }
    const outline = [
      x0, y0, 0, x0 + w, y0, 0,
      x0 + w, y0, 0, x0 + w, y0 + d, 0,
      x0 + w, y0 + d, 0, x0, y0 + d, 0,
      x0, y0 + d, 0, x0, y0, 0,
    ];

    const lines = (points: number[], material: LineBasicMaterial): LineSegments => {
      const geometry = new BufferGeometry();
      geometry.setAttribute('position', new Float32BufferAttribute(points, 3));
      return new LineSegments(geometry, material);
    };
    // Un poco por debajo de Z = 0 para que la rejilla no «pelee» con la base de la pieza.
    this.bedGroup.position.z = -0.05;
    this.bedGroup.add(lines(minor, this.minorMaterial), lines(major, this.majorMaterial), lines(outline, this.outlineMaterial));
  }
}

```

### src/viewer/zoom.ts · 17 líneas

```

/** Cada pulsación acerca o aleja un 15 % (la rueda del ratón hace algo parecido por «muesca»). */
export const KEY_ZOOM_STEP = 1.15;

/**
 * Factor por el que hay que multiplicar la distancia de la cámara al objetivo para una tecla del
 * visor: menor que 1 acerca (`+`, `=`), mayor que 1 aleja (`-`, `_`). Devuelve `null` si no es una
 * tecla de zoom, si lleva Ctrl, Cmd o Alt (son el zoom de la página y atajos del sistema) o si no hay
 * pieza cargada (`hasPart`: con el visor vacío no hay nada que acercar).
 * El cambio es inmediato, sin animación, así que respeta `prefers-reduced-motion`.
 */
export function keyZoomFactor(event: Pick<KeyboardEvent, 'key' | 'ctrlKey' | 'metaKey' | 'altKey'>, hasPart: boolean):
  number | null {
  if (!hasPart || event.ctrlKey || event.metaKey || event.altKey) return null;
  if (event.key === '+' || event.key === '=') return 1 / KEY_ZOOM_STEP;
  if (event.key === '-' || event.key === '_') return KEY_ZOOM_STEP;
  return null;
}

```

## src/ui/app.ts · 781 líneas

```
import { browserLanguages, detectLang, getLang, LANGS, onLangChange, setLang, t, type Key, type Lang } from '../i18n';
import { applyStaticTranslations } from '../i18n/dom';
import { buildQuoteDocument } from '../pdf/document';
import { formatDuration, formatEuro, formatNumber } from '../quote/format';
import { BUSINESS_LIMITS, nextQuoteNumber, normalizeBusiness, type BusinessProfile } from '../quote/business';
import { isMaterialId, MATERIALS } from '../quote/materials';
import { computeQuote, type Quote } from '../quote/model';
import { applyPrinter, CUSTOM_PRINTER, getPrinter, PRINTERS, printerNote } from '../quote/printers';
import { DEFAULT_SETTINGS, LIMITS, normalizeSettings, type QuoteSettings } from '../quote/settings';
import { parseHash, settingsToHash } from '../quote/share';
import { buildQuoteText } from '../quote/text';
import { StlAnalyzer } from '../stl/analyzer';
import { meshWarnings, type MeshWarning } from '../stl/geometry';
import { ModelParseError } from '../stl/errors';
import type { Mesh, MeshStats, Vec3 } from '../stl/types';
import type { Viewer, ViewerTheme } from '../viewer/viewer';
import { supportsWebGL } from '../viewer/webgl';
import { formatLabel } from './format-label';
import { NumberField } from './number-field';
import { renderPrintSheet } from './print-sheet';
import { setupHistory } from './history';
import { LogoError, prepareLogo } from './logo';
import {
  clearBusiness,
  clearSettings,
  loadBusiness,
  loadLang,
  loadSettings,
  saveBusiness,
  saveLang,
  saveSettings,
} from './storage';

const SAMPLE_FILE = 'soporte-movil.stl';
const MAX_FILE_BYTES = 300 * 1024 * 1024;

interface LoadedPart {
  readonly fileName: string;
  readonly mesh: Mesh;
  readonly stats: MeshStats;
}

function byId<T extends HTMLElement> = HTMLElement>(id: string): T {
  const node = document.getElementById(id);
  if (!node) throw new Error(`Falta el elemento #${id} en index.html`);
  return node as T;
}

function readTheme(): ViewerTheme {
  const style = getComputedStyle(document.documentElement);
  const token = (name: string, fallback: string): string => style.getPropertyValue(name).trim() || fallback;
  return {
    background: token('--stage', '#eeeeee7'),
    ink: token('--ink', '#111111'),
    line: token('--line', '#d9d6cf'),
    accent: token('--accent', '#ff5a1f'),
  };
}

function warningText(warning: MeshWarning, size: Vec3, bed: Vec3, largePolygons: number): string {
  switch (warning) {
    case 'open-mesh':
      return t('warn.open');
    case 'inverted':
      return t('warn.inverted');
    case 'big-polygons':
      return t('warn.bigPolygons', { count: formatNumber(largePolygons, 0) });
    case 'tiny':
      return t('warn.tiny', { size: formatNumber(Math.max(size.x, size.y, size.z), 3) });
    case 'too-big':
      return t('warn.tooBig', {
        size: `${formatNumber(size.x, 1)} × ${formatNumber(size.y, 1)} × ${formatNumber(size.z, 1)}`,
        bed: `${formatNumber(bed.x, 0)} × ${formatNumber(bed.y, 0)} × ${formatNumber(bed.z, 0)}`,
      });
  }
}
```

```

/** Quita el listener de idioma de la instancia anterior (los tests arrancan la app varias veces). */
let detachLanguage: (() => void) | null = null;
/** Ídem para el listener de `hashchange`. */
let detachHash: (() => void) | null = null;

/** Descarga un archivo generado en el navegador. */
function download(blob: Blob, fileName: string): void {
  const url = URL.createObjectURL(blob);
  const link = document.createElement('a');
  link.href = url;
  link.download = fileName;
  document.body.append(link);
  link.click();
  link.remove();
  window.setTimeout(() => URL.revokeObjectURL(url), 10_000);
}

async function copyText(text: string): Promise<boolean> {
  try {
    await navigator.clipboard.writeText(text);
    return true;
  } catch {
    // Respaldo para contextos sin la API del portapapeles.
    const area = document.createElement('textarea');
    area.value = text;
    area.setAttribute('readonly', '');
    area.style.position = 'fixed';
    area.style.opacity = '0';
    document.body.append(area);
    area.select();
    try {
      return document.execCommand('copy');
    } catch {
      return false; // sin `execCommand` (o bloqueado): se avisa de que no se ha copiado
    } finally {
      area.remove();
    }
  }
}

/** Arranca la aplicación: enlaza controles, visor, arrastrar y soltar y almacenamiento. */
export function startApp(): void {
  const stage = byId<'viewer'>.parentElement as HTMLElement;
  const notice = byId<'notice'>;
  const live = byId<'live-status'>;
  const loading = byId<'stage-loading'>;
  const fileInput = byId<HTMLInputElement>(<'file-input'>);
  const resetCameraButton = byId<HTMLButtonElement>(<'reset-camera'>);
  const copyButton = byId<HTMLButtonElement>(<'copy-button'>);
  const printButton = byId<HTMLButtonElement>(<'print-button'>);
  const pdfButton = byId<HTMLButtonElement>(<'pdf-button'>);
  const infillRange = byId<HTMLInputElement>(<'in-infill-range'>);
  const materialRadios = Array.from(document.querySelectorAll<HTMLInputElement>(<'input[name="material"]'>));
  const out = (id: string): HTMLElement => byId(<'out-${id}'>);
  /** Solo toca el DOM si el texto cambia: el total es una región viva y no debe repetirse. */
  const setOut = (id: string, text: string): void => {
    const node = out(id);
    if (node.textContent !== text) node.textContent = text;
  };

  let settings = loadSettings();
  let business = loadBusiness();
  let pdfBusy = false;
  let part: LoadedPart | null = null;
  /** Se aplicaron los parámetros de un enlace y aún no hay pieza: el estado vacío lo dice. */
  let sharedApplied = false;
  let quote: Quote | null = null;
  let viewer: Viewer | null = null;
  /** Se crea más abajo, cuando existen `announce`, `showNotice` y `applySettings`. */
  let history: ReturnType<typeof setupHistory> | null = null;
  /** El aviso se guarda como función para poder volver a pintarlo si cambia el idioma. */
  let noticeBuilder: (() => readonly [title: string, message: string]) | null = null;

  const analyzer = new StlAnalyzer();
  /** Cada carga recibe un número; si llega otra antes de terminar, el resultado viejo se descarta. */
  let loadTicket = 0;
  /** Archivo que se está leyendo (para volver a rotular el indicador si cambia el idioma). */

```

```

let loadingName: string | null = null;

// — Visor —
// three.js (~500 kB) se descarga aparte, después de pintar la interfaz: el panel
// y el presupuesto funcionan desde el primer momento aunque el visor tarde.
const noViewer = (): void => showNotice(() => [t('viewer.none.title'), t('viewer.none.text')]);
if (supportsWebGL()) {
  import('../viewer/viewer')
    .then(({ Viewer }) => {
      const container = byId('viewer');
      viewer = new Viewer(container, readTheme());
      // El visor admite teclado (lo gestiona OrbitControls): se hace enfocable y se explica.
      container.tabIndex = 0;
      container.setAttribute('role', 'application');
      applyViewerLabels();
      if (part) viewer.setPart(part.mesh.positions);
      render();
    })
    .catch(() => {
      viewer = null;
      noViewer();
    });
} else {
  noViewer();
}

/** Etiquetas del visor en el idioma activo (solo si el visor ya existe: es cuando se hace enfocable). */
function applyViewerLabels(): void {
  const container = byId('viewer');
  if (!container.hasAttribute('role')) return;
  container.setAttribute('aria-roledescription', t('viewer.roledescription'));
  container.setAttribute('aria-label', t('viewer.label'));
}
window.matchMedia('(prefers-color-scheme: dark)').addEventListener('change', () => viewer?.setTheme(readTheme()));

// — Campos —
const bed = (): Vec3 => ({ x: settings.bedX, y: settings.bedY, z: settings.bedZ });

const update = (patch: Partial<QuoteSettings>): void => {
  settings = normalizeSettings({ ...settings, ...patch });
  saveSettings(settings);
  render();
};

type FieldKey = Exclude<keyof QuoteSettings, 'material' | 'printerId' | 'pricePerKg'>;
const fieldSpec: ReadonlyArray<readonly [inputId: string, errorId: string, key: FieldKey | 'price', decimals: number, step: number]> = [
  ['in-price', 'err-price', 'price', 2, 1],
  ['in-infill', 'err-infill', 'infillPercent', 1, 5],
  ['in-perimeters', 'err-perimeters', 'perimeters', 0, 1],
  ['in-linewidth', 'err-linewidth', 'linewidth', 2, 0.05],
  ['in-flow', 'err-flow', 'flowRate', 1, 1],
  ['in-bedx', 'err-bed', 'bedX', 0, 10],
  ['in-bedy', 'err-bed', 'bedY', 0, 10],
  ['in-bedz', 'err-bed', 'bedZ', 0, 10],
  ['in-power', 'err-power', 'powerWatts', 0, 10],
  ['in-energy', 'err-energy', 'energyPrice', 3, 0.01],
  ['in-margin', 'err-margin', 'marginPercent', 1, 5],
  ['in-copies', 'err-copies', 'copies', 0, 1],
];

const fields = new Map<FieldKey | 'price', NumberField>();
for (const [inputId, errorId, key, decimals, step] of fieldSpec) {
  const limit = key === 'price' ? LIMITS.pricePerKg : LIMITS[key];
  const field = new NumberField({
    input: byId<HTMLInputElement>(inputId),
    error: byId(errorId),
    min: limit.min,
    max: limit.max,
    integer: limit.integer ?? false,
    decimals,
    step,
    onValue: (value) => {
      if (key === 'price') {
        update({ pricePerKg: { ...settings.pricePerKg, [settings.material]: value } });
      } else {
        update({ [key]: value });
      }
    }
  });
}

```

```

    },
  });
  fields.set(key, field);
}

// Perfiles de impresora: elegir uno rellena caudal, potencia y cama; editar cualquiera de
// esos campos vuelve a «Personalizada» (lo hace `normalizeSettings` al ver que ya no coinciden).
const printerSelect = byId<HTMLSelectElement>('in-printer');
const printerNoteEl = byId('printer-note');
const option = (label: string, value: string): HTMLOptionElement => {
  const node = document.createElement('option');
  node.value = value;
  node.textContent = label;
  return node;
};
/** (Re)crea las opciones: «Personalizada» cambia con el idioma; los nombres de modelo no. */
const fillPrinterOptions = (): void => {
  printerSelect.replaceChildren(
    option(t('printer.custom'), CUSTOM_PRINTER),
    ...PRINTERS.map((printer) => option(printer.name, printer.id)),
  );
  printerSelect.value = settings.printerId;
};
fillPrinterOptions();
/**
 * Aplica unos ajustes a todos los campos y recalcula. Por defecto los guarda como los de trabajo; los de un
 * enlace no (`persist = false`): abrir un enlace no pisa lo que la persona tenía guardado.
 */
const applySettings = (next: QuoteSettings, persist = true): void => {
  settings = normalizeSettings(next);
  if (persist) saveSettings(settings);
  syncForm();
  render();
};
printerSelect.addEventListener('change', () => {
  applySettings(applyPrinter(settings, printerSelect.value));
  const printer = getPrinter(settings.printerId);
  announce(printer ? t('printer.applied', { name: printer.name }) : t('printer.custom.applied'));
});

const syncForm = (): void => {
  for (const [key, field] of fields) {
    field.set(key === 'price' ? settings.pricePerKg[settings.material] : settings[key]);
  }
  for (const radio of materialRadios) radio.checked = radio.value === settings.material;
  infillRange.value = String(settings.infillPercent);
  syncPrinter();
};

function syncPrinter(): void {
  printerSelect.value = settings.printerId;
  const printer = getPrinter(settings.printerId);
  printerNoteEl.textContent = printer ? `${t('printer.note.prefix')} ${printerNote(printer)}` : t('printer.note.custom');
}

for (const radio of materialRadios) {
  radio.addEventListener('change', () => {
    if (radio.checked && isMaterialId(radio.value)) {
      update({ material: radio.value });
      fields.get('price')?.set(settings.pricePerKg[settings.material]);
    }
  });
}

infillRange.addEventListener('input', () => {
  const value = Number(infillRange.value);
  fields.get('infillPercent')?.set(value);
  update({ infillPercent: value });
});

byId('reset-settings').addEventListener('click', () => {
  clearSettings();
  settings = DEFAULT_SETTINGS;
  syncForm();
  render();
  announce(t('settings.reset.done'));
});

```

```

});

// — Render —
function render(): void {
  syncPrinter();
  const b = bed();
  viewer?.setBed(b);
  byId('stage-legend').textContent = t('stage.legend', { x: formatNumber(b.x, 0), y: formatNumber(b.y, 0), z:
formatNumber(b.z, 0) });
  infillRange.value = String(settings.infillPercent);
  infillRange.setAttribute('aria-valuetext', `${formatNumber(settings.infillPercent, 0)} %`);

  const hasPart = part !== null;
  stage.classList.toggle('has-part', hasPart);
  resetCameraButton.disabled = !hasPart || !viewer;
  copyButton.disabled = !hasPart;
  printButton.disabled = !hasPart;
  if (!pdfBusy) pdfButton.disabled = !hasPart;

  const warningsList = byId('warnings');
  if (!part) {
    quote = null;
    for (const id of ['volume', 'area', 'size', 'triangles', 'printed', 'weight', 'time', 'material', 'energy',
'subtotal', 'margin', 'total']) {
      setOut(id, '-');
    }
    setOut('kwh', '');
    setOut('margin-pct', '');
    setOut('summary', t(sharedApplied ? 'summary.shared' : 'summary.empty'));
    byId('file-name').textContent = t('stage.none');
    byId('file-detail').textContent = '';
    document.title = t('meta.title');
    byId('stage-dims').textContent = '';
    warningsList.replaceChildren();
    history?.sync();
    return;
  }

  const { stats } = part;
  const size = stats.bounds.size;
  quote = computeQuote(stats, settings);

  setOut('volume', formatNumber(stats.volume / 1000, 2));
  setOut('area', formatNumber(stats.surfaceArea / 100, 2));
  setOut('size', `${formatNumber(size.x, 1)} × ${formatNumber(size.y, 1)} × ${formatNumber(size.z, 1)}`);
  setOut('triangles', formatNumber(stats.triangleCount, 0));

  const dims = byId('stage-dims');
  dims.replaceChildren(
    ...(['x', 'y', 'z'] as const).map((axis) => {
      const span = document.createElement('span');
      const label = document.createElement('b');
      label.textContent = axis.toUpperCase();
      span.append(label, `${formatNumber(size[axis], 1)}`);
      return span;
    }),
    'mm',
  );

  // La lista es una región viva: solo se reescribe si los avisos cambian, para que el
  // lector de pantalla no los repita cada vez que se toca un ajuste.
  const largePolygons = part.mesh.largePolygons ?? 0;
  const warningTexts = meshWarnings(stats, b, largePolygons).map((warning) => warningText(warning, size, b,
largePolygons));
  const currentTexts = Array.from(warningsList.children, (li) => li.textContent ?? '');
  if (warningTexts.join('|') !== currentTexts.join('|')) {
    warningsList.replaceChildren(
      ...warningTexts.map((text) => {
        const li = document.createElement('li');
        li.textContent = text;
        return li;
      }),
    );
  }

  setOut('printed', formatNumber(quote.printedVolume / 1000, 2));
  setOut('weight', formatNumber(quote.totalWeightGrams, 1));

```

```

    setOut('weight-sub', quote.copies > 1 ? t('breakdown.weight.copies', { n: formatNumber(quote.copies, 0) }) : t(
'breakdown.weight.total'));
    setOut('time', formatDuration(quote.totalHours));
    setOut('material', formatNumber(quote.materialCost, 2));
    setOut('energy', formatNumber(quote.energyCost, 2));
    setOut('kwh', `${formatNumber(quote.energyKwh, 2)} kWh`);
    setOut('subtotal', formatNumber(quote.subtotal, 2));
    setOut('margin-pct', `${formatNumber(settings.marginPercent, 0)} %`);
    setOut('margin', formatNumber(quote.marginAmount, 2));
    setOut('total', formatEuro(quote.total));

    const copiesText =
      quote.copies === 1
        ? t('summary.copies.one')
        : t('summary.copies.other', { n: formatNumber(quote.copies, 0), price: formatEuro(quote.totalPerCopy) });
    out('summary').textContent = t('summary.line', {
      copies: copiesText,
      material: MATERIALS[settings.material].name,
      infill: formatNumber(settings.infillPercent, 0),
      time: formatDuration(quote.totalHours),
    });
    byId('file-name').textContent = part.fileName;
    byId('file-detail').textContent = t('file.detail', {
      format: formatLabel(part.mesh.format),
      n: formatNumber(part.mesh.triangleCount, 0),
    });
    document.title = `${part.fileName} · printquote`;
    history?.sync();
  }

  // — Carga de archivos —
  function showNotice(build: () => readonly [title: string, message: string]): void {
    noticeBuilder = build;
    const [title, message] = build();
    const heading = document.createElement('p');
    heading.className = 'notice-title';
    heading.textContent = title;
    const body = document.createElement('p');
    body.textContent = message;
    notice.replaceChildren(heading, body);
    notice.setAttribute('role', 'alert');
    notice.hidden = false;
  }

  function hideNotice(): void {
    noticeBuilder = null;
    notice.hidden = true;
    notice.removeAttribute('role');
    notice.replaceChildren();
  }

  let announceTimer = 0;
  function announce(message: string): void {
    live.textContent = '';
    window.clearTimeout(announceTimer);
    announceTimer = window.setTimeout(() => (live.textContent = message), 50);
  }

  async function loadBuffer(fileName: string, getBuffer: () => Promise<ArrayBuffer>): Promise<void> {
    const ticket = ++loadTicket;
    hideNotice();
    stage.classList.add('is-loading');
    stage.setAttribute('aria-busy', 'true');
    loadingName = fileName;
    loading.textContent = t('load.reading', { name: fileName });
    loading.hidden = false;
    announce(t('load.reading.announce', { name: fileName }));
    try {
      const buffer = await getBuffer();
      // El parseo y la geometría van en un Web Worker: la interfaz sigue respondiendo.
      const { mesh, stats } = await analyzer.analyze(buffer, fileName);
      if (ticket !== loadTicket) return; // ya se ha pedido otro archivo
      part = { fileName, mesh, stats };
      viewer?.setPart(mesh.positions);

      render();
      // El total se anuncia solo: su contenedor es una región viva.
    }
  }

```

```

        announce(t('load.done', { name: fileName }));
    } catch (error) {
        if (ticket !== loadTicket) return;
        showNotice(() => [
            t('load.failed.title', { name: fileName }),
            error instanceof ModelParseError ? t(error.code, error.params) : t('err.unexpected'),
        ]);
    } finally {
        if (ticket === loadTicket) {
            stage.classList.remove('is-loading');
            stage.removeAttribute('aria-busy');
            loading.hidden = true;
            loadingName = null;
        }
    }
}

function loadFile(file: File): void {
    if (file.size > MAX_FILE_BYTES) {
        showNotice(() => [t('load.tooBig.title', { name: file.name }), t('load.tooBig.text')]);
        return;
    }
    void loadBuffer(file.name, () => file.arrayBuffer());
}

byId('open-button').addEventListener('click', () => fileInput.click());
fileInput.addEventListener('change', () => {
    const file = fileInput.files?.[0];
    if (file) loadFile(file);
    fileInput.value = '';
});

const loadSample = (): void => {
    void loadBuffer(SAMPLE_FILE, async () => {
        const response = await fetch(`${import.meta.env.BASE_URL}samples/${SAMPLE_FILE}`);
        if (!response.ok) throw new ModelParseError('err.sample');
        return response.arrayBuffer();
    });
};
byId('sample-button').addEventListener('click', loadSample);

// — Arrastrar y soltar en toda la ventana —
const overlay = byId('drop-overlay');
let dragDepth = 0;
const hasFiles = (event: DragEvent): boolean => Array.from(event.dataTransfer?.types ?? []).includes('Files');

window.addEventListener('dragenter', (event) => {
    if (!hasFiles(event)) return;
    event.preventDefault();
    dragDepth++;
    overlay.hidden = false;
});
window.addEventListener('dragover', (event) => {
    if (!hasFiles(event)) return;
    event.preventDefault();
    if (event.dataTransfer) event.dataTransfer.dropEffect = 'copy';
});
window.addEventListener('dragleave', (event) => {
    if (!hasFiles(event)) return;
    dragDepth = Math.max(0, dragDepth - 1);
    if (dragDepth === 0) overlay.hidden = true;
});
window.addEventListener('drop', (event) => {
    if (!hasFiles(event)) return;
    event.preventDefault();
    dragDepth = 0;
    overlay.hidden = true;
    const file = event.dataTransfer?.files[0];
    if (file) loadFile(file);
});

// — Acciones —
resetCameraButton.addEventListener('click', () => viewer?.resetCamera());

let copyLabelTimer = 0;
copyButton.addEventListener('click', () => {
    if (!part || !quote) return;

```

```

const text = buildQuoteText({ fileName: part.fileName, stats: part.stats, settings, quote });
void copyText(text).then((ok) => {
  // La etiqueta se restablece desde el diccionario (no desde el texto actual): con dos clics
  // seguidos se quedaba en «Copiado».
  copyButton.textContent = t(ok ? 'copy.done' : 'copy.failed');
  announce(t(ok ? 'copy.announce.ok' : 'copy.announce.failed'));
  window.clearTimeout(copyLabelTimer);
  copyLabelTimer = window.setTimeout(() => (copyButton.textContent = t('action.copy')), 1800);
});
});

const fillPrintSheet = (): void => {
  if (!part || !quote) return;
  renderPrintSheet(byId('print-sheet'), {
    fileName: part.fileName,
    format: part.mesh.format,
    stats: part.stats,
    settings,
    quote,
    image: viewer ? viewer.snapshot() : null,
  });
};
window.addEventListener('beforeprint', fillPrintSheet);
printButton.addEventListener('click', () => {
  fillPrintSheet();
  window.print();
});

// — Datos del negocio y presupuesto en PDF —
const bizText = [
  ['in-biz-name', 'name'],
  ['in-biz-taxid', 'taxId'],
  ['in-biz-address', 'address'],
  ['in-biz-phone', 'phone'],
  ['in-biz-email', 'email'],
  ['in-biz-web', 'web'],
  ['in-biz-number', 'quoteNumber'],
] as const;
const bizInputs = bizText.map(([id, key]) => [byId<HTMLInputElement | HTMLTextAreaElement>(id), key] as const);
const logoInput = byId<HTMLInputElement>('in-biz-logo');
const logoPreview = byId('logo-preview');
const logoImg = byId<HTMLImageElement>('logo-img');
const logoStatus = byId('logo-status');
/** Mensaje actual bajo el logotipo (una clave, para poder traducirlo si cambia el idioma). */
let logoStatusKey: Key = 'biz.logo.hint';
const setLogoStatus = (key: Key): void => {
  logoStatusKey = key;
  logoStatus.textContent = t(key);
};

const updateBusiness = (patch: Partial<BusinessProfile>): boolean => {
  business = normalizeBusiness({ ...business, ...patch });
  return saveBusiness(business);
};

const syncBusiness = (): void => {
  for (const [input, key] of bizInputs) input.value = business[key];
  validityField.set(business.validityDays);
  vatField.set(business.vatPercent);
  logoPreview.hidden = business.logo === null;
  if (business.logo) logoImg.src = business.logo;
  else logoImg.removeAttribute('src');
};

for (const [input, key] of bizInputs) {
  input.addEventListener('input', () => updateBusiness({ [key]: input.value }));
  // Al salir del campo se muestra lo que de verdad se ha guardado (recortado, o el número por defecto si se vació).
  input.addEventListener('change', () => (input.value = business[key]));
}

const validityField = new NumberField({
  input: byId<HTMLInputElement>('in-biz-validity'),
  error: byId('err-validity'),
  min: BUSINESS_LIMITS.validityDays.min,
  max: BUSINESS_LIMITS.validityDays.max,
  integer: true,
  decimals: 0,
});

```

```

    step: 1,
    onValue: (value) => void updateBusiness({ validityDays: value }),
  });
const vatField = new NumberField({
  input: byId<HTMLInputElement>('in-biz-vat'),
  error: byId('err-vat'),
  min: BUSINESS_LIMITS.vatPercent.min,
  max: BUSINESS_LIMITS.vatPercent.max,
  decimals: 2,
  step: 1,
  onValue: (value) => void updateBusiness({ vatPercent: value }),
});

logoInput.addEventListener('change', () => {
  const file = logoInput.files?.[0];
  logoInput.value = '';
  if (!file) return;
  setLogoStatus('biz.logo.processing');
  prepareLogo(file)
    .then((logo) => {
      const saved = updateBusiness({ logo });
      syncBusiness();
      setLogoStatus(saved ? 'biz.logo.saved' : 'biz.logo.notSaved');
    })
    .catch((error: unknown) => {
      setLogoStatus(error instanceof LogoError ? error.code : 'err.logo.generic');
    });
});

byId('logo-remove').addEventListener('click', () => {
  updateBusiness({ logo: null });
  syncBusiness();
  setLogoStatus('biz.logo.removed');
  logoInput.focus();
});

byId('reset-business').addEventListener('click', () => {
  clearBusiness();
  business = normalizeBusiness(undefined);
  syncBusiness();
  setLogoStatus('biz.logo.hint');
  announce(t('biz.reset.done'));
});

byId('settings-form').addEventListener('submit', (event) => event.preventDefault());

const pdfFileName = (): string => `${t('pdf.title').toLowerCase()}-${business.quoteNumber.replace(/[^\A-Za-z0-9._-]+/g, '-')}.pdf`;

pdfButton.addEventListener('click', () => {
  if (!part || !quote || pdfBusy) return;
  pdfBusy = true;
  pdfButton.disabled = true;
  pdfButton.textContent = t('pdf.generating');
  announce(t('pdf.generating.announce'));
  const current = { part, quote, settings, business };
  void (async () => {
    try {
      // pdf-lib y fontkit (~1,1 MB; ~500 kB gzip) y las fuentes solo se descargan la primera vez que se pide un PDF.
      const { renderQuotePdf } = await import('../pdf/render');
      const bytes = await renderQuotePdf(
        buildQuoteDocument({
          business: current.business,
          fileName: current.part.fileName,
          stats: current.part.stats,
          settings: current.settings,
          quote: current.quote,
          image: viewer ? viewer.snapshot() : null,
        })
      );
      const name = pdfFileName();
      download(new Blob([bytes as BlobPart], { type: 'application/pdf' }), name);
      // El número sube solo tras descargar: el siguiente presupuesto ya sale con el nuevo.
      updateBusiness({ quoteNumber: nextQuoteNumber(current.business.quoteNumber) });
      syncBusiness();
      announce(t('pdf.done', { name, next: business.quoteNumber }));
    } catch (error) {
      console.error(error);
      // Sin conexión (o sin las fuentes) es un fallo distinto de un error al dibujar el documento.
    }
  })();
});

```

```

        const fonts = error instanceof Error && error.name === 'FontLoadError';
        showNotice(() => [t('pdf.failed.title'), t(fonts ? 'pdf.failed.fonts' : 'pdf.failed.text')]);
    } finally {
        pdfBusy = false;
        pdfButton.textContent = t('action.pdf');
        pdfButton.disabled = !part;
    }
    })();
});

// — Historial de presupuestos —
history = setupHistory({
    current: () => (part && quote ? { fileName: part.fileName, stats: part.stats, quote, settings } : null),
    applySettings,
    announce,
    showNotice: (title, message) => showNotice(() => [title, message]),
    download,
});

// — Enlace con los parámetros (en el hash de la URL) —
const shareButton = byId<HTMLButtonElement>('share-copy');
let shareLabelTimer = 0;
shareButton.addEventListener('click', () => {
    // Origen y ruta de esta página + el hash con los ajustes actuales (sin pieza, sin cliente, sin negocio).
    const link = `${window.location.origin}${window.location.pathname}${settingsToHash(settings)}`;
    void copyText(link).then((ok) => {
        shareButton.textContent = t(ok ? 'share.copied' : 'share.copyFailed');
        announce(t(ok ? 'share.copied' : 'share.copyFailed'));
        window.clearTimeout(shareLabelTimer);
        shareLabelTimer = window.setTimeout(() => (shareButton.textContent = t('share.copy')), 1800);
    });
});

/** `#v=1&...` aplica los parámetros; `#ejemplo` (solo o con ellos) carga la pieza de ejemplo. */
const applyHash = (): void => {
    const { settings: shared, example } = parseHash(window.location.hash, settings);
    if (shared) {
        applySettings(shared, false);
        sharedApplied = part === null;
        render();
        announce(t(part ? 'share.applied.part' : 'share.applied'));
    }
    if (example) loadSample();
};
detachHash?.();
window.addEventListener('hashchange', applyHash);
detachHash = () => window.removeEventListener('hashchange', applyHash);

// — Idioma —
const langButtons = Array.from(document.querySelectorAll<HTMLButtonElement>('button[data-lang]'));

/** Vuelve a pintar todo lo que depende del idioma: HTML estático, campos, avisos y cifras. */
function applyLanguage(): void {
    const lang = getLang();
    document.documentElement.lang = lang;
    applyStaticTranslations();
    for (const node of document.querySelectorAll<HTMLElement>('[data-density]')) {
        const id = node.dataset['density'];
        if (isMaterialId(id)) node.textContent = formatNumber(MATERIALS[id].density, 2);
    }
    for (const button of langButtons) button.setAttribute('aria-pressed', String(button.dataset['lang'] === lang));
    fillPrinterOptions();
    logoStatus.textContent = t(logoStatusKey);
    if (loadingName !== null) loading.textContent = t('load.reading', { name: loadingName });
    // El último anuncio para lectores de pantalla quedaría en el idioma anterior.
    window.clearTimeout(announceTimer);
    live.textContent = '';
    applyViewerLabels();
    syncForm();
    syncBusiness();
    history?.render();
    shareButton.textContent = t('share.copy');
    if (noticeBuilder) showNotice(noticeBuilder);
    if (!pdfBusy) pdfButton.textContent = t('action.pdf');
    render();
}
for (const button of langButtons) {

```

```

button.addEventListener('click', () => {
  const lang = button.dataset['lang'];
  if (lang && (LANGS as readonly string[]).includes(lang)) {
    saveLang(lang as Lang);
    setLang(lang as Lang);
  }
});
}

detachLanguage?.();
setLang(detectLang(browserLanguages(), loadLang()));
detachLanguage = onLangChange(applyLanguage);
applyLanguage();

// Enlaces: .../printquote/#ejemplo (la pieza de ejemplo) y .../printquote/#v=1&... (parámetros compartidos).
applyHash();
}

```

## src/i18n/index.ts · 76 líneas

```
import { en } from './en';
import { es, type ErrorKey, type Key } from './es';
import { interpolate, type Params } from './interpolate';

export type { ErrorKey, Key, Params };

export type Lang = 'es' | 'en';
export const LANGS: readonly Lang[] = ['es', 'en'];

const DICTIONARIES: Readonly<Record<Lang, Readonly<Record<Key, string>>>> = { es, en };

/** Configuración regional de `Intl` de cada idioma: números, moneda y fechas. La moneda sigue siendo el euro. */
const LOCALES: Readonly<Record<Lang, string>> = { es: 'es-ES', en: 'en-GB' };

let current: Lang = 'es';
const listeners = new Set<(lang: Lang) => void>();

export function isLang(value: unknown): value is Lang {
  return value === 'es' || value === 'en';
}

export function getLang(): Lang {
  return current;
}

/** Cambia el idioma activo y avisa a quien lo escuche. */
export function setLang(lang: Lang): void {
  if (lang === current) return;
  current = lang;
  for (const listener of listeners) listener(lang);
}

/** Escucha los cambios de idioma. Devuelve la función para dejar de escuchar. */
export function onLangChange(listener: (lang: Lang) => void): () => void {
  listeners.add(listener);
  return () => listeners.delete(listener);
}

/** Configuración regional de `Intl` del idioma activo (o del indicado). */
export function getLocale(lang: Lang = current): string {
  return LOCALES[lang];
}

/** Traduce una clave a un idioma concreto. */
export function tIn(lang: Lang, key: Key, params?: Params): string {
  return interpolate(DICTIONARIES[lang][key], params);
}

/** Traduce una clave al idioma activo. */
export function t(key: Key, params?: Params): string {
  return tIn(current, key, params);
}

/**
 * Idioma inicial: el guardado si lo hay; si no, el primero de los idiomas del navegador que
 * sea español o inglés. Si el navegador no declara ninguno de los dos, inglés (lo entiende más
 * gente); si no declara nada, español.
 */
export function detectLang(navigatorLanguages: readonly string[], saved?: string | null): Lang {
  if (isLang(saved)) return saved;
  if (navigatorLanguages.length === 0) return 'es';
  for (const tag of navigatorLanguages) {
    const primary = tag.toLowerCase().split(/[-_]/)[0];
    if (primary === 'es' || primary === 'en') return primary;
  }
  return 'en';
}

/** Idiomas que declara el navegador (vacío si no hay `navigator`). */
export function browserLanguages(): readonly string[] {
  if (typeof navigator === 'undefined') return [];
  const list = navigator.languages;
  if (list && list.length > 0) return list;
  return navigator.language ? [navigator.language] : [];
}
```

`.github/workflows/ci.yml` · 42 líneas

```
name: CI

on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-{{ github.ref }}
  cancel-in-progress: true

jobs:
  check:
    name: Lint, tests y build ({{ matrix.os }})
    runs-on: {{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Tests
        run: npm test

      - name: Build
        run: npm run build
```

## .github/workflows/deploy.yml · 53 líneas

```
name: Despliegue en GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Tests
        run: npm test

      - name: Build (base /printquote/)
        run: npm run build

      - uses: actions/configure-pages@v5

      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Publicar en Pages
        id: deployment
        uses: actions/deploy-pages@v4
```